计算机网络面试 100 题

从协议原理到 Linux 网络栈与线上故障定位

题解式深度面试宝典 · 参考“问题驱动、层层追问”的教材组织方式

本书定位

不是“背诵 100 个结论”，而是训练一条完整的网络推理链：

应用协议 → 传输协议 → IP/路由 → Linux 内核 → 抓包证据 → 业务故障。

每题统一包含：30 秒回答、深度解析、追问树、易错点、动手实验与评分标准。

2026 Edition

适用：后端 / C++ / Go / Java / 基础架构 / SRE / 网络开发

使用说明：如何像“剑指 Offer”一样练网络题

- 本书借鉴的是“以问题为入口、通过追问逐层暴露知识盲点”的教材方法，而不是复制任何受版权保护的原文、题解或版式。
- 一轮练习不要直接读“深度解析”。先口述 30 秒答案，再按追问树自测；如果能画图、写状态机、给出抓包证据，才算真正掌握。
- 大厂网络题的区分度越来越来自工程细节：send 成功不等于对端处理、TIME_WAIT 不是内核垃圾、HTTP/2 仍有 TCP HOL、epoll ET 要读到 EAGAIN、DNS 不只使用 UDP。

推荐答题结构

1. 先给结论：30 秒内回答“是什么”。
2. 再讲机制：状态、数据结构、时序、公式。
3. 再讲 Why：为什么这样设计，不这样会怎样。
4. 最后工程化：Linux 命令、抓包、故障指标与边界情况。

全书知识地图

章节	题号	核心能力	面试权重
第一章 网络分层、链路与二层基础	1-10	建立“逐层封装与当前一跳”的基本心智模型。	★★★☆☆
第二章 IP、NAT 与路由	11-20	从本机路由选择扩展到自治系统与全局流量调度。	★★★★☆
第三章 TCP 连接、状态机与可靠性	21-35	面试最高频核心：握手、挥手、TIME_WAIT、序号、重传与 framing。	★★★★★
第四章 TCP 性能、窗口与拥塞控制	36-45	理解吞吐为什么受到 RTT、窗口、拥塞算法和系统队列共同约束。	★★★★☆
第五章 UDP、QUIC 与 HTTP/3 传输基础	46-52	从“无连接数据报”到现代用户态可靠传输协议。	★★★☆☆
第六章 DNS、全局调度与 CDN	53-60	掌握域名解析、缓存、一致性与边缘加速。	★★★★☆
第七章 HTTP/1.1、HTTP/2 与应用层协议	61-73	从 HTTP 语义深入到 framing、缓存、多路复用与 RPC。	★★★★★
第八章 HTTPS、TLS 1.3 与 PKI	74-80	掌握握手、证书、密钥协商与 0-RTT 风险。	★★★★★
第九章 Socket、epoll 与 Linux	81-90	把协议知识落到系统调用、事件	★★★★★

章节	题号	核心能力	面试权重
网络栈		循环与内核收包路径。	
第十章 综合故障定位与系统设计	91-100	用分层方法处理真实线上问题与高并发架构题。	★★★★★

目录

第一章 网络分层、链路与二层基础

建立“逐层封装与当前一跳”的基本心智模型。

01. OSI 七层与 TCP/IP 模型如何对应？工程上为什么更常用 TCP/IP?
02. 一个应用层消息从 send() 到网卡发送经历了什么封装?
03. Hub、Switch、Router 的本质区别是什么?
04. MAC 地址和 IP 地址为什么要同时存在?
05. ARP 的工作原理是什么？跨网段通信时 ARP 查询谁?
06. MTU 与 MSS 有什么区别？为什么常见 MSS 是 1460?
07. 子网掩码 / CIDR 前缀到底解决什么问题?
08. ping 的完整原理是什么？ping 不通能否说明 TCP 也不通?
09. traceroute 为什么能看到中间路由器?
10. 为什么二层需要 VLAN？VLAN 与广播域是什么关系?

第二章 IP、NAT 与路由

从本机路由选择扩展到自治系统与全局流量调度。

11. NAT 到底做了什么？SNAT、DNAT、PAT 有什么区别?
12. NAT 为什么让 P2P 变难？STUN、TURN、ICE 分别做什么?
13. DHCP 的 DORA 流程是什么？为什么客户端一开始没有 IP 也能通信?
14. IPv4 与 IPv6 除了地址长度，还有哪些关键差异?
15. IP Fragmentation 为什么通常应该避免？Path MTU Discovery 如何工作?
16. TTL / Hop Limit 为什么存在？路由环路是怎么形成的?
17. 路由器为什么使用最长前缀匹配?
18. OSPF 与 BGP 的定位和核心思想有什么不同?
19. BGP 的 AS_PATH、LOCAL_PREF、MED 分别解决什么?
20. Anycast 是什么？为什么 DNS/CDN 常用?

第三章 TCP 连接、状态机与可靠性

面试最高频核心：握手、挥手、TIME_WAIT、序号、重传与 framing。

21. TCP 与 UDP 的本质区别是什么？不要只说“可靠/不可靠”。
22. TCP 三次握手完整过程是什么？每一步同步了什么状态?
23. 为什么 TCP 需要三次握手，不能两次?
24. 第三次握手 ACK 丢失后，客户端立刻带数据发送，服务端能建立连接吗?
25. TCP 为什么通常需要四次挥手？FIN 与 ACK 的语义是什么?
26. TIME_WAIT 为什么存在？为什么经常讨论 2MSL?
27. 服务器几十万个 TIME_WAIT，应该怎么分析而不是直接改内核参数?
28. CLOSE_WAIT 大量堆积通常说明什么?
29. 什么是 TCP Half-close？什么时候有用?
30. SYN Flood 怎么攻击？SYN Cookies 是什么，为什么不是万能方案?
31. TCP Sequence Number 与 ACK Number 到底在确认什么?
32. TCP 可靠性由哪些机制共同保证?
33. 超时重传、快速重传和 SACK 分别解决什么?
34. TCP 滑动窗口解决什么问题？发送窗口为什么不是固定大小?
35. 所谓 TCP “粘包/拆包”到底是什么？应用层怎样正确 framing?

第四章 TCP 性能、窗口与拥塞控制

理解吞吐为什么受到 RTT、窗口、拥塞算法和系统队列共同约束。

36. Flow Control 与 Congestion Control 的区别是什么?
37. Slow Start 为什么叫“慢启动”，它的增长其实有多快?
38. Fast Retransmit 与 Fast Recovery 分别是什么?
39. rwnd、cwnd、ssthresh 三者是什么关系?
40. TCP Reno 与 CUBIC 的核心差异是什么?
41. BBR 与 CUBIC 的核心思想差在哪里?
42. Nagle Algorithm 解决什么？为什么交互式 RPC 常设置 TCP_NODELAY?
43. TCP_NODELAY 到底改变什么，不改变什么?
44. BDP 是什么？为什么高带宽长 RTT 链路需要大窗口?
45. TCP 半连接队列、全连接队列与 listen(backlog) 有什么关系?

第五章 UDP、QUIC 与 HTTP/3 传输基础

从“无连接数据报”到现代用户态可靠传输协议。

46. 如果基于 UDP 实现可靠传输，需要补哪些机制?
47. UDP 为什么不存在 TCP 那种“粘包”语义?
48. QUIC 为什么建立在 UDP 上，而不是直接改 TCP?
49. HTTP/2 已有多路复用，为什么 HTTP/3 仍需要 QUIC?
50. QUIC Connection Migration 为什么适合移动网络?
51. QUIC 为什么能减少首次/恢复连接延迟？0-RTT 的收益与风险是什么?
52. QUIC 使用 UDP，是否就不需要拥塞控制和流量控制?

第六章 DNS、全局调度与 CDN

掌握域名解析、缓存、一致性与边缘加速。

53. 输入域名后，DNS 完整解析链路是什么?
54. DNS 递归查询与迭代查询有什么区别?
55. 为什么“DNS 使用 UDP”是错误或不完整答案?
56. A、AAAA、CNAME、MX、NS、TXT 分别表达什么?
57. DNS TTL 与 Negative Caching 是什么?
58. DNS 如何做负载均衡？它的局限是什么?
59. DNS Cache Poisoning 是什么？DNSSEC、DoH/DoT 分别保护什么?
60. CDN 为什么能加速？缓存命中率为什么只是其中一部分?

第七章 HTTP/1.1、HTTP/2 与应用层协议

从 HTTP 语义深入到 framing、缓存、多路复用与 RPC。

61. HTTP/1.1 Request/Response 报文结构是什么?
62. GET 与 POST 真正的区别是什么？safe、idempotent、cacheable 如何区分?
63. 常见 HTTP 状态码中，301/302/307/308、401/403、502/504 怎么区分?
64. HTTP “无状态”是什么意思？Cookie、Session、Token 为什么出现?
65. HTTP Persistent Connection 与 TCP 长连接是什么关系?
66. Content-Length 与 chunked transfer encoding 解决什么?
67. Cache-Control 中 max-age、no-cache、no-store、private、must-revalidate 怎么理解?
68. ETag/If-None-Match 与 Last-Modified/If-Modified-Since 如何工作?
69. HTTP/1.1 与 HTTP/2 的核心差异是什么?

70. HTTP/2 为什么仍存在 Head-of-Line Blocking?
71. HPACK 与 QPACK 为什么不能完全相同?
72. WebSocket、SSE、Long Polling 应该怎么选?
73. gRPC 为什么常运行在 HTTP/2 上?

第八章 HTTPS、TLS 1.3 与 PKI

掌握握手、证书、密钥协商与 0-RTT 风险。

74. HTTP 与 HTTPS 的本质区别是什么？TLS 提供了哪些安全属性?
75. TLS 1.3 Handshake 需要掌握到什么程度?
76. 浏览器如何验证 HTTPS 证书？证书链、主机名、有效期各自是什么作用?
77. TLS 为什么同时使用非对称密码学和对称加密?
78. 什么是 Forward Secrecy？为什么 ECDHE 很重要?
79. SNI 与 ALPN 分别解决什么问题?
80. TLS 1.3 0-RTT 为什么有 Replay Risk？哪些业务不适合?

第九章 Socket、epoll 与 Linux 网络栈

把协议知识落到系统调用、事件循环与内核收包路径。

81. 一个 TCP Server 从 socket() 到处理请求的系统调用流程是什么?
82. Blocking Socket 与 Non-blocking Socket 的核心区别是什么?
83. 经典五种 I/O Model 是什么？readiness 与 completion 的区别是什么?
84. select、poll、epoll 的核心差异是什么?
85. epoll LT 与 ET 有什么区别？ET 为什么要求“读到 EAGAIN”？
86. 为什么 epoll ET 几乎总与 nonblocking socket 搭配?
87. 为什么 epoll 对普通磁盘文件意义有限?
88. send() 返回成功，是否意味着对端已经收到或处理?
89. Zero Copy 的目标是什么？sendfile、splice、mmap 各自大致减少了什么?
90. 一个 Packet 从 NIC 到应用 recv() 大致经过哪些 Linux 网络栈阶段?

第十章 综合故障定位与系统设计

用分层方法处理真实线上问题与高并发架构题。

91. 浏览器输入 https://www.example.com 到页面显示，完整发生了什么?
92. 用户说“网站突然很慢”，你如何系统化拆解而不是猜?
93. 服务延迟升高但 CPU、QPS 都没变化，你优先怀疑什么?
94. 大量 TIME_WAIT 的完整排障流程是什么?
95. 大量 CLOSE_WAIT 的完整排障流程是什么?
96. 大量 SYN_RECV 且新连接失败，应如何判断是攻击、丢包还是容量问题?
97. 网络服务 CPU 很高但业务线程不忙，可能是什么?
98. 四层负载均衡与七层负载均衡怎么选?
99. 设计支持 100 万 TCP 长连接的服务器，第一性约束有哪些?
100. 给你一份 tcpdump/Wireshark 抓包，如何系统判断 TCP 问题?

第一章 网络分层、链路与二层基础

本章目标

建立“逐层封装与当前一跳”的基本心智模型。

建议：先能口述结论，再能画出关键时序/状态，最后通过命令或抓包验证。

01. OSI 七层与 TCP/IP 模型如何对应？工程上为什么更常用 TCP/IP？

难度： 基础　**频率：** ★★★★★　**标签：** 分层 / 封装 / 协议栈　**定位：** 核心真题

30 秒标准回答

OSI 更适合教学分层；工程中通常用应用层、传输层、网络层、链路层（再加物理层）理解实际协议栈。关键不是背层数，而是能解释 HTTP 数据如何逐层封装为 TCP Segment、IP Packet、Ethernet Frame。

核心时序 / 心智模型

```
Application Data
      ↓
TCP Segment
      ↓
IP Packet
      ↓
Ethernet Frame
      ↓
Bits on medium
```

深度解析：Know-How + Know-Why

- 分层的核心价值是职责隔离：上层只依赖下层提供的服务接口，协议可独立演进。
- 发送端完成“应用数据→传输层首部→IP 首部→链路层首部/尾部”的封装；接收端反向解封装。
- OSI 的表示层/会话层在互联网工程里通常由应用协议或库承担，因此 TCP/IP 模型更贴近真实实现。

面试官常见追问

- TLS 属于哪一层？为什么不能机械套 OSI？
- 路由器、交换机分别主要处理哪一层信息？
- 一个 HTTP 请求经过 NAT 后哪些头字段可能变化？

高频易错点

- 把“七层”背出来却说不清数据单位。
- 认为每一层在操作系统中都对应一个独立进程。

动手验证 / 抓包实验

用 Wireshark 抓一次 HTTPS 请求，分别观察 Ethernet、IP、TCP、TLS 四层头部；确认同一帧中各层如何嵌套。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

02. 一个应用层消息从 send() 到网卡发送经历了什么封装？

难度：基础　频率：★★★★☆　标签：封装 / 内核网络栈　定位：核心真题

30 秒标准回答

应用写入 socket 后，内核把字节交给 TCP/UDP，加入传输层头；再经过 IP 路由与分片/PMTU 决策，加入 IP 头；链路层解析下一跳并构造帧，最后由 NIC 通过 DMA/队列发送。

深度解析：Know-How + Know-Why

- TCP 是字节流：一次 send() 不等于一个 TCP Segment；分段由 MSS、拥塞窗口、TSO 等共同影响。
- IP 层选择出口设备和下一跳；同网段发目标主机，跨网段通常发默认网关。
- 链路层目的 MAC 是“当前下一跳”的 MAC，而不是最终服务器 MAC。

面试官常见追问

- 为什么 send() 成功不等于网卡已经发出?
- TSO/GSO 会怎样影响抓包看到的包大小?
- 路由器转发时是否重新构造二层帧?

高频易错点

- 把用户态 send() 与物理发送混为一谈。
- 认为应用每次 write 的边界会完整保留到 TCP 对端。

动手验证 / 抓包实验

Linux 上用 `strace -e sendto,write` 与 `tcpdump -i any` 对照，观察系统调用次数与实际 TCP Segment 数量不一致。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

03. Hub、Switch、Router 的本质区别是什么？

难度：基础　频率：★★★★☆　标签：L1 / L2 / L3　定位：核心真题

30 秒标准回答

Hub 主要做物理层信号复制；交换机基于 MAC 表在二层转发 Frame；路由器基于 IP 前缀和路由表在三层选择下一跳。三者最本质的区别是“基于什么信息做转发决策”。

深度解析：Know-How + Know-Why

- 交换机通过源 MAC 学习 MAC→Port 映射，未知目的 MAC 会在 VLAN 内泛洪。
- 路由器进行最长前缀匹配，改变每一跳的二层头，并递减 IPv4 TTL/IPv6 Hop Limit。
- 三层交换机是把大量 L3 转发能力放进交换 ASIC，并不改变 IP 路由原理。

面试官常见追问

- 交换机为什么会广播 ARP?
- VLAN 如何限制二层广播域?
- 路由器为什么不转发普通二层广播?

高频易错点

- 把 Switch 说成“不会广播”。
- 把 Router 说成“根据 MAC 地址转发”。

动手验证 / 抓包实验

在一个两网段实验拓扑中执行 `arp -n`、`ip route`，分别观察主机邻居表与路由表。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

04. MAC 地址和 IP 地址为什么要同时存在？

难度：基础　**频率：**★★★★★　**标签：**寻址 / 下一跳　**定位：**核心真题

30 秒标准回答

IP 解决跨网络的逻辑寻址与路由；MAC 解决某个二层网络中当前一跳的帧交付。跨多个路由器时，目标 IP 通常保持为最终主机，而每一跳的源/目的 MAC 都会重写。

深度解析：Know-How + Know-Why

- 主机先用路由表判断目标是否在本地子网。

- 同网段时 ARP/NDP 解析目标主机；跨网段时解析默认网关。
- 以太网无法直接把帧“送到互联网任意 IP”，它只负责本地链路的一跳。

面试官常见追问

- 访问公网服务器时 ARP 查询公网 IP 还是网关 IP?
- NAT 会不会改变最终目标 IP?
- MAC 地址是否全球一定唯一?

高频易错点

- 认为目标 MAC 一路保持不变。
- 把 MAC 当成“物理位置地址”，忽略虚拟网卡、容器、MAC 重写等情况。

动手验证 / 抓包实验

抓取跨网段 ping：在客户端侧看目的 MAC 是网关；在下一跳链路上，二层地址已改变。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

05. ARP 的工作原理是什么？跨网段通信时 ARP 查询谁？

难度：基础	频率：★★★★★	标签：ARP / 邻居解析	定位：核心真题

30 秒标准回答

ARP 用 IPv4 地址解析本地二层邻居的 MAC。若目标与本机同网段，就解析目标主机；若跨网段，则解析路由表选出的下一跳（通常是默认网关），绝不会靠 ARP 直接解析远端互联网主机。

深度解析：Know-How + Know-Why

- ARP Request 常以二层广播发送，Reply 通常单播返回。
- 解析结果进入邻居缓存，避免每个数据包都广播查询。
- ARP 只在同一个二层广播域中有效，路由器不会把普通 ARP 广播转发到远端网络。

面试官常见追问

- 什么是 Gratuitous ARP?
- ARP Spoofing 怎么发生?
- IPv6 为什么不用 ARP，而用 NDP?

高频易错点

- “访问 8.8.8.8 就 ARP 8.8.8.8”是经典错误。
- 把 ARP 缓存与 DNS 缓存混淆。

动手验证 / 抓包实验

运行 `ip neigh flush all` 后 ping 同网段主机与公网地址，配合 tcpdump 观察分别查询目标主机和网关。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

06. MTU 与 MSS 有什么区别？为什么常见 MSS 是 1460？

难度：进阶　**频率：**★★★★★　**标签：**MTU / MSS / 分段　**定位：**核心真题

30 秒标准回答

MTU 是链路层可承载的最大网络层 Packet 大小；MSS 是 TCP 连接中协商的最大 TCP Payload。典型以太网 MTU 1500、IPv4 无选项 20B、TCP 无选项 20B，因此常见 MSS≈1460，但真实值受 IPv6、隧道、VPN、TCP 选项和路径 MTU 影响。

深度解析：Know-How + Know-Why

- MSS 由 TCP SYN 中的 MSS option 通告，用于避免生成过大的 TCP Segment。
- IPv6 基础头 40B，因此同一 MTU 下可用 TCP payload 与 IPv4 不同。
- 隧道会增加额外封装，导致“物理链路 MTU 1500，但内层可用 MTU 更小”。

面试官常见追问

- MSS 与 Path MTU 是什么关系?
- 为什么 VPN 常见 MTU 1400/1420?
- 抓包为什么偶尔看到超过 1500 字节的 TCP Segment?

高频易错点

- 把 MTU 当成“应用一次最多 send 的字节数”。
- 认为 1460 是所有网络的固定 MSS。

动手验证 / 抓包实验

执行 `ip link` 查看 MTU；用 `ping -M do -s <size>` 探索无分片上限；抓 SYN 观察 MSS option。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

07. 子网掩码 / CIDR 前缀到底解决什么问题？

难度：基础	频率：★★★★☆	标签：CIDR / 子网	定位：核心真题

30 秒标准回答

CIDR 前缀把 IP 地址拆成“网络前缀 + 主机部分”，主机据此判断目标是否属于本地网络，路由器也据此前缀聚合路由。/24 表示前 24 位是网络前缀，而不是“固定能用 254 台机器”的同义词。

深度解析：Know-How + Know-Why

- 主机通过 `(dst & mask) == (local & mask)` 判断本地子网。
- CIDR 允许可变长度前缀，避免传统 A/B/C 类地址的严重浪费。
- 路由汇总能减少全局路由表规模。

面试官常见追问

- /31、/32 有什么用途?
- 为什么云厂商一个 /24 未必能分配 254 个用户地址?
- 最长前缀匹配如何利用 CIDR?

高频易错点

- 机械套用 `2^hostbits - 2` 到所有现代网络场景。
- 把子网掩码与默认网关混为一谈。

动手验证 / 抓包实验

用 Python `ipaddress` 模块验证 /20、/27 的网络地址、广播地址与 host 范围。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

08. ping 的完整原理是什么？ping 不通能否说明 TCP 也不通？

难度：基础	频率：★★★★☆	标签：ICMP / 诊断	定位：核心真题

30 秒标准回答

ping 通常发送 ICMP Echo Request 并等待 Echo Reply，用于测试 IP 层可达性和 RTT。ICMP 可能被防火墙丢弃，因此 ping 不通不等于 TCP/443 不通；反过来 ping 通也不代表应用服务正常。

深度解析：Know-How + Know-Why

- 域名 ping 前通常先触发 DNS。
- ICMP 是网络层控制/错误报告协议，不通过 TCP 建连。
- RTT 高可能来自路径、排队、无线链路，也不等价于应用响应慢。

面试官常见追问

- 为什么某些云主机 ping 不通但 HTTPS 正常?
- ICMP Destination Unreachable 有哪些用途?
- ping 的 RTT 与 TCP RTT 一定一致吗?

高频易错点

- 说 ping 使用 TCP。
- 把 ping 当作应用健康检查的充分条件。

动手验证 / 抓包实验

对同一主机分别执行 `ping`、`curl -w`、`nc -vz host 443`，比较三种“可达性”含义。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

09. traceroute 为什么能看到中间路由器？

难度：进阶 **频率：**★★★★☆ **标签：**TTL / ICMP / 路径诊断 **定位：**核心真题

30 秒标准回答

traceroute 利用 IP TTL/Hop Limit：先发 TTL=1 的探测包，第一跳减为 0 后返回 ICMP Time Exceeded；再逐步增加 TTL，从而枚举路径。不同实现可用 UDP、ICMP 或 TCP 探测，所以结果会受 ACL、ECMP、NAT 和设备策略影响。

深度解析：Know-How + Know-Why

- TTL 的主要目的之一是避免路由环路导致数据包无限循环。
- 某跳显示 `*` 不等于该路由器不存在，可能只是它不回 ICMP。
- ECMP 可能让不同探测包走不同路径，使 traceroute 显示的路径并非单一固定链路。

面试官常见追问

- 为什么 traceroute 去程路径不等于回程路径?
- mtr 比 traceroute 多提供了什么?
- 为什么基于 TCP/443 的 traceroute 有时更有用?

高频易错点

- 把 TTL 理解成严格的“秒数寿命”。
- 把每一跳 RTT 直接当作该路由器处理耗时。

动手验证 / 抓包实验

分别用 `traceroute`、`traceroute -T -p 443`、`mtr` 比较同一目标，观察防火墙与路径差异。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

10. 为什么二层需要 VLAN？VLAN 与广播域是什么关系？

难度：进阶　**频率：**★★★☆☆　**标签：**VLAN / 二层隔离　**定位：**核心真题

30 秒标准回答

VLAN 在同一物理交换网络上划分逻辑二层广播域。不同 VLAN 的广播默认不会互通，跨 VLAN 通信需要三层路由。它解决广播范围、故障隔离、租户/安全分区和网络组织问题。

深度解析：Know-How + Know-Why

- 802.1Q 通过 VLAN Tag 标识帧所属 VLAN。
- Access Port 通常承载单一 VLAN，Trunk Port 可承载多个 VLAN。
- VLAN 是二层隔离机制，不等价于完整安全边界；ACL、路由、防火墙仍然重要。

面试官常见追问

- 同一交换机上不同 VLAN 为什么不能直接 ARP?
- Native VLAN 有什么风险?
- VXLAN 与 VLAN 的关系是什么?

高频易错点

- 认为 VLAN 能替代防火墙。
- 把 VLAN ID 当作跨互联网可路由的全局标识。

动手验证 / 抓包实验

在 Linux network namespace + bridge 环境创建两个 VLAN，验证同 VLAN 可二层互通、跨 VLAN 需路由。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

第二章 IP、NAT 与路由

本章目标

从本机路由选择扩展到自治系统与全局流量调度。

建议：先能口述结论，再能画出关键时序/状态，最后通过命令或抓包验证。

11. NAT 到底做了什么？SNAT、DNAT、PAT 有什么区别？

难度：进阶 **频率：**★★★★★ **标签：**NAT / 端口映射 **定位：**核心真题

30 秒标准回答

NAT 改写报文中的 IP 地址，PAT/NAPT 还改写端口，并维护连接映射状态。SNAT 常用于内网访问公网；DNAT 常用于公网入口映射到内网服务；家庭路由器最常见的是多个私网连接共享一个公网 IP 的 NAPT。

深度解析：Know-How + Know-Why

- NAT 设备通常需要同步更新 IP/TCP/UDP 校验和。
- 状态型 NAT 依赖连接跟踪，因此会消耗表项并可能成为性能瓶颈。
- NAT 打破端到端寻址，使 P2P、入站连接、故障定位更复杂。

面试官常见追问

- NAT 表满会有什么现象?
- 四元组/五元组如何用于 NAT 映射?
- NAT 与反向代理的区别是什么?

高频易错点

- 把 NAT 说成“只改 IP，不改端口”。
- 认为 NAT 天然等价于安全防火墙。

动手验证 / 抓包实验

用 `iptables/nftables` 做一次 SNAT/DNAT 实验，并查看 conntrack 表。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

12. NAT 为什么让 P2P 变难？STUN、TURN、ICE 分别做什么？

难度：进阶　**频率：**★★★☆☆　**标签：**NAT Traversal / WebRTC　**定位：**核心真题

30 秒标准回答

NAT 后的私网端点不可从公网直接路由，且不同 NAT 的映射/过滤行为不同。STUN 帮客户端发现公网映射；TURN 在无法直连时提供中继；ICE 是一套候选地址收集、连通性检查与路径选择框架。

深度解析：Know-How + Know-Why

- UDP hole punching 依赖双方先向外发送，借助 NAT 已建立的映射。
- 对称 NAT 等类型会显著降低直连成功率。
- TURN 可靠但成本高，因为媒体流量经过中继服务器。

面试官常见追问

- 为什么 WebRTC 不直接只用 STUN?
- NAT 类型会如何影响 hole punching?
- IPv6 普及后 NAT traversal 是否完全消失?

高频易错点

- 把 STUN 当成中继服务。
- 认为 ICE 是一个单独传输协议。

动手验证 / 抓包实验

使用公开 WebRTC ICE candidate 日志观察 host、srflx、relay 三类候选。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

13. DHCP 的 DORA 流程是什么？为什么客户端一开始没有 IP 也能通信？

难度：基础　**频率：**★★★★☆　**标签：**DHCP / 地址分配　**定位：**核心真题

30 秒标准回答

DHCP 经典流程为 Discover→Offer→Request→ACK。客户端初始还没有可用地址，因此通过广播和特殊源/目的地址获取配置；服务端可下发 IP、Mask、Gateway、DNS、Lease 等。

深度解析：Know-How + Know-Why

- DHCP 基于 UDP，客户端/服务端使用固定端口。
- 租约机制使地址可复用，客户端会在租期内提前续租。
- 跨网段 DHCP 常通过 DHCP Relay，把广播请求转发到集中服务器。

面试官常见追问

- DHCP Relay 为什么需要?
- 租约到期会怎样?
- 静态 IP 与 DHCP Reservation 有何差别?

高频易错点

- 把 DORA 当成 TCP 握手。
- 认为 DHCP 只下发 IP，不下发 DNS/Gateway。

动手验证 / 抓包实验

抓取一次 DHCP renew 或在虚拟机重启网卡，观察 Discover/Offer/Request/ACK。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

14. IPv4 与 IPv6 除了地址长度，还有哪些关键差异？

难度：进阶 **频率：**★★★★☆ **标签：**IPv6 / NDP / Header **定位：**核心真题

30 秒标准回答

IPv6 使用 128 位地址、固定基础头、扩展头体系、NDP 替代 ARP，并取消路由器中途分片；多播承担许多 IPv4 广播场景。其目标不只是“地址更多”，还包括更简洁的转发与更好的协议扩展性。

深度解析：Know-How + Know-Why

- IPv6 基础头没有 IPv4 header checksum，减少逐跳处理。
- Fragment Extension Header 由源端添加，路由器不会替源主机分片。
- NDP 基于 ICMPv6 完成邻居解析、路由器发现、地址自动配置等。

面试官常见追问

- IPv6 为什么不需要广播?
- SLAAC 和 DHCPv6 如何区别?
- 双栈环境为什么会出现 Happy Eyeballs?

高频易错点

- 认为 IPv6 天生自带加密。
- 认为 IPv6 完全不需要任何 NAT 场景。

动手验证 / 抓包实验

执行 `ip -6 addr`、`ip -6 neigh`、`ping -6`，抓取 Neighbor Solicitation/Advertisement。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

15. IP Fragmentation 为什么通常应该避免？Path MTU Discovery 如何工作？

难度：进阶	频率：★★★★★	标签：分片 / PMTU	定位：核心真题

30 秒标准回答

IP 分片会放大丢包代价、增加重组状态和中间设备兼容问题。现代传输更倾向让源端根据 Path MTU 控制报文大小。IPv4 可通过 DF + ICMP Fragmentation Needed，IPv6 则由路由器返回 Packet Too Big。

深度解析：Know-How + Know-Why

- 一个 fragment 丢失就会使原始 IP datagram 无法完整重组。
- 中间防火墙若错误丢弃必要 ICMP，可能导致 PMTU black hole。
- TCP 常结合 MSS 控制，尽量不让正常 Segment 触发 IP fragmentation。

面试官常见追问

- 什么是 PMTU black hole?
- 为什么 “ping 通但大包 TCP 卡住” 可能与 MTU 有关?
- IPv6 的最小链路 MTU 有什么意义?

高频易错点

- 认为 TCP 自身会做“IP 分片”。
- 把 TCP segmentation 与 IP fragmentation 当成同一件事。

动手验证 / 抓包实验

用不同 payload 的 `ping -M do` 查最大无分片尺寸；再通过 VPN/隧道场景观察 PMTU 变化。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

16. TTL / Hop Limit 为什么存在？路由环路是怎么形成的？

难度：基础 **频率：**★★★★☆ **标签：**TTL / 路由环路 **定位：**核心真题

30 秒标准回答

TTL/Hop Limit 为数据包设置最大跳数，防止由于配置错误、收敛暂态等产生的路由环路让报文无限循环。每经过一个三层转发节点都会递减，到 0 即丢弃并通常返回 ICMP Time Exceeded。

深度解析：Know-How + Know-Why

- TTL 是协议层的“生命上限”，不是应用超时。
- 路由协议在拓扑变化时可能短暂产生不一致状态。
- traceroute 正是借助 TTL 逐跳到期来探测路径。

面试官常见追问

- 为什么 TTL 到 0 不是发送 TCP RST?
- 操作系统默认 TTL 为什么不同?
- 环路会怎样消耗链路带宽?

高频易错点

- 把 TTL 当成毫秒。
- 认为交换机也一定会递减 IP TTL。

动手验证 / 抓包实验

抓取 traceroute 包，检查每一轮 probe 的 TTL。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

17. 路由器为什么使用最长前缀匹配？

难度：进阶 **频率：**★★★★★ **标签：**Routing / LPM **定位：**核心真题

30 秒标准回答

一个目的地址可能同时匹配默认路由、汇总路由和更具体路由。最长前缀匹配选择最具体的可达规则，使路由既能聚合又能做局部精细控制。例如 10.1.2.3 同时匹配 /8、/16、/24 时选择 /24。

深度解析：Know-How + Know-Why

- 路由控制平面生成 RIB，转发平面通常使用优化后的 FIB。
- 默认路由 `0.0.0.0/0` 能匹配所有 IPv4 目的，但优先级最低。
- ECMP 可以让同一前缀存在多条等价下一跳。

面试官常见追问

- 行政距离、metric 与最长前缀谁先决定?
- FIB 与 RIB 的区别?
- ECMP 如何避免单个 flow 乱序?

高频易错点

- 看到更低 metric 就忽略前缀长度。
- 把路由表理解成按从上到下第一条匹配。

动手验证 / 抓包实验

用 `ip route get <dst>` 查看内核对具体目标的下一跳决策。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

18. OSPF 与 BGP 的定位和核心思想有什么不同？

难度：进阶　**频率：**★★★★☆　**标签：**IGP / EGP / 路由协议　**定位：**核心真题

30 秒标准回答

OSPF 是典型 IGP，服务自治系统内部的链路状态路由与最短路径计算；BGP 是域间路径向量/策略路由协议，核心是可扩展的前缀传播和商业/策略控制，而不是简单“全网最短路”。

深度解析：Know-How + Know-Why

- OSPF 通过 LSA 构造链路状态数据库，再运行 SPF。
- BGP 通过 UPDATE 传播 NLRI 与路径属性，路由选择高度受 LOCAL_PREF、AS_PATH、MED 等策略影响。
- 互联网自治系统之间需要可表达商业关系的策略，这正是 BGP 的核心价值。

面试官常见追问

- 为什么不能用 OSPF 直接跑整个互联网?

- BGP 收敛为什么通常比 IGP 慢?
- iBGP 与 eBGP 有什么差别?

高频易错点

- 把 BGP 说成“Dijkstra 的互联网版本”。
- 认为 AS_PATH 最短永远最高优先级。

动手验证 / 抓包实验

阅读本机/实验路由器的 OSPF/BGP 路由表，比较同一前缀的 next-hop 与属性。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

19. BGP 的 AS_PATH、LOCAL_PREF、MED 分别解决什么？

难度：进阶 **频率：**★★★☆☆ **标签：**BGP 属性 / 策略 **定位：**核心真题

30 秒标准回答

AS_PATH 记录经过的自治系统并帮助环路检测；LOCAL_PREF 是 AS 内部常用的出站偏好，数值通常越高越优先；MED 向邻居表达“进入我方 AS 的偏好入口”，通常越低越优先，但实际选择受厂商与策略配置影响。

深度解析：Know-How + Know-Why

- BGP 路由选择是多阶段策略链，而非只看一个属性。
- AS_PATH 还能用于 prepending，人工降低某条入站路径吸引力。
- LOCAL_PREF 不应直接当作跨 AS 统一可比较的“全局成本”。

面试官常见追问

- 为什么 BGP 策略常常违反物理最短路径?
- Community 属性有什么用?
- Route leak 与 prefix hijack 有什么区别?

高频易错点

- 把 MED 和 LOCAL_PREF 的方向搞反。
- 把所有属性当作全网统一含义。

动手验证 / 抓包实验

使用 FRRouting/Containerlab 构建 3 AS 小拓扑，通过修改 local-pref 和 AS-path prepend 观察选路变化。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

20. Anycast 是什么？为什么 DNS/CDN 常用？

难度：进阶　**频率：**★★★☆☆　**标签：**Anycast / CDN / BGP　**定位：**核心真题

30 秒标准回答

Anycast 让多个物理节点同时宣布同一 IP 前缀，BGP 根据现有路由策略把用户流量送往某个实例。它能降低平均网络距离、分散 DDoS 流量、提高服务冗余，但“最近”是 BGP 意义上的最佳路径，并非地理最近。

深度解析：Know-How + Know-Why

- 公共 DNS 常用 Anycast 将全球用户导向附近 PoP。
- 故障节点撤销路由后，流量会逐步收敛到其他节点。
- 长连接场景要考虑路由变化导致的路径漂移与状态一致性。

面试官常见追问

- Anycast 与 DNS-based GSLB 有什么区别?
- 为什么 Anycast 不保证最低 RTT?
- Anycast TCP 服务如何处理状态?

高频易错点

- 认为 Anycast 是一个“负载均衡器设备”。
- 把 BGP 最优路径等价于物理距离最近。

动手验证 / 抓包实验

对公共 Anycast DNS 从不同网络/云区域 traceroute，比较同一 IP 到达不同 PoP 的可能性。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

第三章 TCP 连接、状态机与可靠性

本章目标

面试最高频核心：握手、挥手、TIME_WAIT、序号、重传与 framing。

建议：先能口述结论，再能画出关键时序/状态，最后通过命令或抓包验证。

21. TCP 与 UDP 的本质区别是什么？不要只说“可靠/不可靠”。

难度：基础	频率：★★★★★	标签：TCP / UDP / 语义	定位：核心真题

30 秒标准回答

TCP 提供面向连接、可靠、有序的字节流，并实现重传、流量控制与拥塞控制；UDP 提供无连接的报文数据报服务，保留消息边界但不保证送达、顺序或去重。应用应依据语义和时延/控制需求选择，而不是简单说“TCP 慢、UDP 快”。

深度解析：Know-How + Know-Why

- TCP 的可靠性来自 ACK、序号、重传、接收缓存等组合。
- UDP 没有内建拥塞控制，但基于 UDP 的上层协议（如 QUIC）必须自己实现。
- TCP 没有消息边界，UDP 保留 datagram 边界。

面试官常见追问

- 实时音视频为什么常倾向 UDP?
- DNS 为什么既可 UDP 也可 TCP?
- QUIC 为什么建立在 UDP 上?

高频易错点

- 说 UDP 一定更快。
- 认为 UDP 不需要任何拥塞控制责任。

动手验证 / 抓包实验

用 `nc -l`/Python 分别写一个 TCP 与 UDP echo，观察多次 send 的边界行为。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

22. TCP 三次握手完整过程是什么？每一步同步了什么状态？

难度： 基础　**频率：** ★★★★★　**标签：** TCP Handshake / ISN　**定位：** 核心真题

> **30 秒标准回答**
>
> 客户端发送 SYN(seq=x)，服务端返回 SYN+ACK(seq=y, ack=x+1)，客户端再 ACK(ack=y+1)。三次握手不仅确认双向路径，也同步双方初始序列号、TCP options，并让两端状态机安全进入 ESTABLISHED。

核心时序 / 心智模型

```
Client                        Server
  | ---- SYN seq=x -----------> |
  | <--- SYN,ACK seq=y -------- |
  | ---- ACK ack=y+1 ---------> |
  |       ESTABLISHED           |
```

深度解析：Know-How + Know-Why

- SYN 本身占用一个序列号，因此 ACK 为 ISN+1。
- SYN 中会协商 MSS、Window Scale、SACK Permitted、Timestamp 等能力。
- 服务端收到首个 SYN 后进入 SYN-RECEIVED，客户端发完 SYN 后处于 SYN-SENT。

面试官常见追问

- 第三次 ACK 丢失会怎样?
- SYN 与普通数据为什么使用同一序列空间?
- TCP Fast Open 改变了什么?

高频易错点

- 只会背 SYN/SYN-ACK/ACK，不知道 seq/ack。
- 说“三次只是为了确认双方收发能力”，没有状态同步视角。

动手验证 / 抓包实验

抓取 `curl` 的 TCP 建连，逐帧标出 Flags、Seq、Ack、MSS、WS、SACK。

> **评分标准**
>
> 及格：能给出核心定义与基本流程。
>
> 良好：能解释关键状态/字段，并回答至少 2 个追问。
>
> 优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

23. 为什么 TCP 需要三次握手，不能两次？

难度： 进阶　**频率：** ★★★★★　**标签：** Handshake / 旧报文　**定位：** 核心真题

30 秒标准回答

两次只能让服务端知道客户端的 SYN 被收到、客户端知道服务端存在，但服务端无法确认客户端已经收到服务端的初始序列信息。第三次 ACK 完成双向 ISN 确认，也帮助避免旧的延迟连接请求让服务端错误建立连接。

深度解析：Know-How + Know-Why

- 握手的本质是双方同步“发送起点”和接收状态。
- 网络可能复制、延迟旧 SYN；状态机必须能识别并避免错误复用旧连接上下文。
- TCP 的可靠全双工语义需要双方都确认对端序列空间。

面试官常见追问

- 为什么四次握手不是必须?
- 如果第二次 SYN+ACK 丢了会怎样?
- 为什么 QUIC 建连逻辑不同?

高频易错点

- 回答“因为 RFC 规定了三次”。
- 用“确认收发能力”一句话结束，无法解释旧 SYN。

动手验证 / 抓包实验

画出“旧 SYN 延迟到达服务端”的两次握手反例，再说明第三次确认如何避免服务端单方面误判。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

24. 第三次握手 ACK 丢失后，客户端立刻带数据发送，服务端能建立连接吗？

难度：进阶　**频率：**★★★★☆　**标签：**TCP 状态机 / ACK+Data　**定位：**核心真题

30 秒标准回答

可以。TCP Segment 可同时携带 ACK 与数据；如果服务端处于 SYN-RECEIVED，收到一个确认其 SYN 的合法 ACK，即使该报文同时携带 payload，也可推进到 ESTABLISHED 并处理数据。第三个“包”并不要求是纯 ACK。

深度解析：Know-How + Know-Why

- 若 ACK 与后续数据均丢失，服务端会按定时器重传 SYN+ACK。
- TCP 状态机关注报文中的控制位和序号合法性，不关注“这是第几个物理包”。
- 应用看到连接建立时，底层可能已经完成若干次重传。

面试官常见追问

- 客户端 connect() 返回成功的时点是什么?
- SYN+ACK 重传次数由什么控制?
- 服务端 SYN queue 中的状态何时转到 accept queue?

高频易错点

- 认为第三次必须是一张“单独 ACK 包”。
- 把 ACK 丢失直接等价为连接一定失败。

动手验证 / 抓包实验

用 tc/netem 或防火墙规则丢弃某次 ACK，抓包观察 SYN+ACK 重传与后续 ACK+Data 行为。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

25. TCP 为什么通常需要四次挥手？FIN 与 ACK 的语义是什么？

难度：基础 **频率：**★★★★★ **标签：**TCP Close / Half-close **定位：**核心真题

30 秒标准回答

TCP 是全双工连接，两个方向可独立关闭。A 的 FIN 只表示“A 不再发送更多字节”，B 仍可能继续发送数据；因此对 A 的 FIN 先 ACK，等 B 自己发送完再发 FIN，最终由 A ACK，典型上表现为四个报文。

深度解析：Know-How + Know-Why

- ACK 可以和 FIN 合并，因此抓包不一定总是严格四个物理报文。
- 收到 FIN 后应用 read() 最终会看到 EOF，但本端仍可写，形成 half-close。
- 主动关闭方在最后 ACK 后通常进入 TIME_WAIT。

面试官常见追问

- 为什么有时只看到三次挥手?
- shutdown(SHUT_WR) 与 close() 有何不同?
- RST 与 FIN 的关闭语义有什么不同?

高频易错点

- 认为 FIN 意味着“双方连接立即消失”。
- 把 TCP 双向字节流当作一条单向通道。

动手验证 / 抓包实验

写一个 socket 程序调用 `shutdown(SHUT_WR)` 后继续 recv，观察 half-close。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

26. TIME_WAIT 为什么存在？为什么经常讨论 2MSL？

难度：进阶 **频率：**★★★★★ **标签：**TIME_WAIT / 连接复用 **定位：**核心真题

30 秒标准回答

TIME_WAIT 主要保护两个问题：第一，保证最后 ACK 丢失时仍能应答对端重传的 FIN；第二，让旧连接中的延迟报文在同一四元组被重新使用前从网络中消失。2MSL 是经典设计中的等待尺度，不应机械理解成所有系统固定秒数。

深度解析：Know-How + Know-Why

- 主动关闭方通常进入 TIME_WAIT，而不是“服务器一定进入”。
- TIME_WAIT 是可靠关闭的一部分，不是简单的资源泄漏。
- 四元组快速复用时尤其要考虑旧报文与新连接序列空间混淆。

面试官常见追问

- 大量 TIME_WAIT 是否一定有问题?
- tcp_tw_reuse 应如何理解?
- 为什么负载均衡/短连接服务特别容易看到 TIME_WAIT?

高频易错点

- 把 TIME_WAIT 当成需要“优化掉”的垃圾状态。
- 说 2MSL 是为了“让连接彻底断开”，没有 ACK/旧报文解释。

动手验证 / 抓包实验

用 `ss -tan state time-wait` 统计主动关闭一侧；做短连接压测，观察连接复用前后 TIME_WAIT 数量。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

27. 服务器几十万个 TIME_WAIT，应该怎么分析而不是直接改内核参数？

难度：工程 **频率：**★★★★★ **标签：**TIME_WAIT / 排障 **定位：**核心真题

30 秒标准回答

先判断谁主动关闭、连接建立速率、远端分布、是否发生短连接风暴和连接池/HTTP keep-alive 失效，再看端口范围、FD、NAT/LB 约束。TIME_WAIT 往往是上游连接生命周期设计的结果，sysctl 只是最后的边界调优。

深度解析：Know-How + Know-Why

- 按 remote endpoint 聚合，能快速发现某个 upstream 是否被频繁重连。
- 客户端侧要关注 ephemeral port 耗尽；服务端侧要关注短连接成本和连接复用。
- 部署、健康检查和代理配置变化也会制造瞬时连接重建。

面试官常见追问

- 如何区分端口耗尽与 FD 耗尽?
- HTTP/2 多路复用为什么能减少 TIME_WAIT?
- NAT 网关为什么也会受短连接影响?

高频易错点

- 第一步就调 `tcp_fin_timeout`。
- 只看 TIME_WAIT 数量，不看连接创建速率和四元组分布。

动手验证 / 抓包实验

收集 `ss -s`、`ss -tan`、连接速率指标；将 TIME_WAIT 按目的 IP:Port 排名并对照连接池配置。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

28. CLOSE_WAIT 大量堆积通常说明什么？

难度：工程　**频率：**★★★★★　**标签：**CLOSE_WAIT / 资源泄漏　**定位：**核心真题

30 秒标准回答

CLOSE_WAIT 表示对端已发送 FIN、内核也已确认，但本地应用尚未 close()。大量长期 CLOSE_WAIT 通常指向应用资源释放路径、异常分支、连接池或线程阻塞问题，而不是 TCP 内核自动“清理不及时”。

深度解析：Know-How + Know-Why

- CLOSE_WAIT 的退出依赖本地应用关闭 socket。

- 如果 FD 大量泄漏，最终可能触发 `EMFILE`/`Too many open files`。
- Java/Go/C++ 都要检查 finally/defer/RAII、HTTP body close、异常路径和连接归还。

面试官常见追问

- CLOSE_WAIT 与 FIN_WAIT_2 有什么对应关系?
- 进程退出后 CLOSE_WAIT 会怎样?
- 怎么定位哪个调用栈持有这些 FD?

高频易错点

- 误把 CLOSE_WAIT 当成 TIME_WAIT 的同类调参问题。
- 只重启服务不修复 close 路径。

动手验证 / 抓包实验

用 `ss -tanp state close-wait` 找 PID/FD，再用 `lsof`、线程栈、应用 tracing 定位未关闭连接。

> **评分标准**
>
> 及格：能给出核心定义与基本流程。
>
> 良好：能解释关键状态/字段，并回答至少 2 个追问。
>
> 优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

29. 什么是 TCP Half-close？什么时候有用？

难度： 进阶 **频率：** ★★★☆☆ **标签：** shutdown / 半关闭 **定位：** 核心真题

> **30 秒标准回答**
>
> TCP 的两个方向可独立关闭。调用 `shutdown(SHUT_WR)` 会发送 FIN，表示本端不再写，但仍可继续 recv 对端剩余数据。这对“发送完请求后继续读取直到 EOF”的协议、代理和流式处理很有价值。

深度解析：Know-How + Know-Why

- close() 通常释放整个 socket 引用，而 shutdown() 可以精确控制读/写方向。
- 收到 FIN 只影响对端的发送方向，并不自动阻止本端继续 send。
- 半关闭是理解 FIN 状态机和四次挥手的关键。

面试官常见追问

- read() 在收到 FIN 后返回什么?
- SO_LINGER 会如何影响 close?
- 为什么 RST 不提供优雅 half-close 语义?

高频易错点

- 把 FIN 理解成“整个连接双向立刻结束”。
- 认为 shutdown 和 close 完全等价。

动手验证 / 抓包实验

实现一个客户端发送请求后 `shutdown(SHUT_WR)`，服务端继续返回大响应，验证客户端仍可接收。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

30. SYN Flood 怎么攻击？SYN Cookies 是什么，为什么不是万能方案？

难度：工程　**频率：**★★★★★　**标签：**SYN Flood / SYN Cookie　**定位：**核心真题

30 秒标准回答

SYN Flood 通过大量不完成握手的 SYN 消耗半连接队列与状态资源。SYN Cookie 在压力下把部分连接状态编码进服务端生成的序列号，减少 SYN_RECV 状态存储，但它是过载防御机制，不应替代正确 backlog、容量规划和上游 DDoS 防护。

深度解析：Know-How + Know-Why

- 攻击可使用伪造源地址，使服务端 SYN+ACK 发向不存在或无关主机。
- 正常高峰也可能把 SYN queue 打满，因此要区分攻击与容量不足。
- 边缘防护、SYN proxy、rate limit、Anycast 等可在更上游吸收攻击。

面试官常见追问

- SYN queue 与 accept queue 有何不同?
- syncookies 打开后是否所有连接都使用 cookie?
- SYN Flood 和 ACK Flood 有何差别?

高频易错点

- 把 SYN Cookie 当成“验证码”。
- 看到 SYN_RECV 多就直接认定 DDoS。

动手验证 / 抓包实验

在隔离实验环境使用 hping3 低速模拟未完成握手，观察 `ss state syn-recv` 和内核 counters，避免对公网目标测试。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

31. TCP Sequence Number 与 ACK Number 到底在确认什么？

难度：基础　**频率：**★★★★★　**标签：**Seq / Ack / Byte Stream　**定位：**核心真题

30 秒标准回答

TCP 序号对字节进行编号，不是给“包”编号。ACK 是累计确认：ACK=N 表示 N 之前的字节已按序接收，下一期望字节是 N。SYN 和 FIN 也占用序列空间。

深度解析：Know-How + Know-Why

- 例如 seq=1000、payload=100B 覆盖 1000~1099，正常累计 ACK 为 1100。
- 重传可复用同一字节序号，使接收端去重并重排。
- 累计 ACK 本身无法精确表达多个离散已收区间，因此 SACK 提供额外区块信息。

面试官常见追问

- 为什么 ACK 可以“跳着涨”？
- 乱序包到达时 ACK 如何表现?
- 为什么抓包工具常显示相对序号而非绝对序号?

高频易错点

- 说 ACK=N 表示“第 N 个包收到”。
- 忽略 SYN/FIN 对序列号的消耗。

动手验证 / 抓包实验

抓一个下载流，在 Wireshark 开启/关闭 relative sequence numbers，手工核对 payload 长度与 ACK 增量。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

32. TCP 可靠性由哪些机制共同保证？

难度：基础　**频率：**★★★★★　**标签：**可靠传输 / 重传 / SACK　**定位：**核心真题

30 秒标准回答

TCP 的可靠性不是单一“重传”，而是序号、ACK、校验、发送/接收缓存、RTO、快速重传、SACK、重排序、流量控制与拥塞控制共同构成。它保证的是有序字节流交付，不保证应用业务一定处理成功。

深度解析：Know-How + Know-Why

- checksum 检测传输错误；序号/ACK 处理丢失、重复和乱序。
- RTO 处理长时间未确认；dup ACK/SACK 提高部分丢包恢复效率。
- socket ACK 只代表对端 TCP stack 接收，不代表数据库事务已提交。

面试官常见追问

- TCP 为什么不能保证“Exactly Once 业务语义”？
- 连接断开后如何判断最后一个请求是否被服务端执行?
- SACK 为什么在高丢包/大窗口下重要?

高频易错点

- 把 TCP 可靠性等同于“网络绝不丢包”。
- 把 TCP ACK 当作业务 ACK。

动手验证 / 抓包实验

设计一个“扣款 RPC”故障场景：客户端发完后连接断开，讨论为什么必须使用幂等键/业务确认。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

33. 超重、快速重和 SACK 分别解决什么？

难度：进阶	频率：★★★★★	标签：RTO / Fast Retransmit / SACK	定位：核心真题

30 秒标准回答

RTO 在 ACK 长时间缺失时兜底重传；经典快速重传利用多个重复 ACK 更早推断缺口，无需等计时器；SACK 告知发送端接收方已收到哪些离散区间，使发送端更精准补齐丢失数据。

深度解析：Know-How + Know-Why

- RTO 过小会产生虚假重传，过大则恢复慢，因此需基于 RTT 估计。
- 重复 ACK 可能来自丢包，也可能来自乱序，所以现代实现还结合 SACK、RACK 等更复杂算法。
- SACK 是 TCP option，需要双方能力支持。

面试官常见追问

- 为什么 3 duplicate ACK 不是数学真理?
- 什么是 spurious retransmission?
- RACK 与经典 dupACK 有何思路差异?

高频易错点

- 认为只要收到 3 个 ACK 就重传。
- 把 SACK 说成“替代累计 ACK”。

动手验证 / 抓包实验

用 `tc netem loss` 在本地命名空间注入少量丢包，抓取 Dup ACK、SACK block 与 Retransmission。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

34. TCP 滑动窗口解决什么问题？发送窗口为什么不是固定大小？

难度：基础 **频率：**★★★★★ **标签：**Sliding Window / Pipeline **定位：**核心真题

30 秒标准回答

滑动窗口允许多个未确认字节同时在途，避免 stop-and-wait 在大 RTT 链路上浪费带宽。发送窗口随 ACK 前移，并同时受接收方 `rwnd` 与拥塞控制 `cwnd` 约束，所以是动态的。

深度解析：Know-How + Know-Why

- 窗口本质上限制“允许 outstanding 的序列空间”。
- Window Scale option 让 TCP 窗口突破 16-bit 头字段的直接上限，适合大 BDP。
- ACK 到来使左边界前移，应用新数据可进入窗口右侧。

面试官常见追问

- Window Scale 为什么必须在握手协商?
- 零窗口时连接怎么恢复?
- 窗口与 socket buffer 有什么关系?

高频易错点

- 把滑动窗口只解释成“批量发包”。
- 认为发送窗口只由接收端决定。

动手验证 / 抓包实验

在 `ss -ti` 中查看 send/receive window、cwnd，并在高 RTT netem 环境观察吞吐与窗口变化。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

35. 所谓 TCP “粘包/拆包”到底是什么？应用层怎样正确 framing？

难度：基础 **频率：**★★★★★ **标签：**Byte Stream / Framing **定位：**核心真题

30 秒标准回答

TCP 只提供连续字节流，不保留应用 write() 边界，因此接收方一次 read 可能拿到半条消息、一条消息或多条消息。所谓“粘包”不是 TCP 错误，而是应用协议没有明确消息边界。

深度解析：Know-How + Know-Why

- 常见 framing：固定长度、分隔符、长度前缀、TLV、自描述协议。
- 长度前缀协议必须防御恶意超大长度，避免内存分配攻击。
- HTTP/1.1 通过 Content-Length/chunked 等规则界定 body；HTTP/2 则用二进制 frame 长度。

面试官常见追问

- UDP 为什么没有同样的粘包语义?
- Protobuf 自身是否自动解决 TCP framing?
- 半包解析器如何设计状态机?

高频易错点

- 试图通过 sleep 或“每次 read 大一点”解决。
- 认为 `send()` 一次对应对端 `recv()` 一次。

动手验证 / 抓包实验

写长度前缀 echo 协议，故意把一个消息拆成多次 send、多个消息合并一次 send，验证解析器仍正确。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

第四章 TCP 性能、窗口与拥塞控制

本章目标

理解吞吐为什么受到 RTT、窗口、拥塞算法和系统队列共同约束。

建议：先能口述结论，再能画出关键时序/状态，最后通过命令或抓包验证。

36. Flow Control 与 Congestion Control 的区别是什么？

难度：基础　频率：★★★★★　标签：rwnd / cwnd　定位：核心真题

30 秒标准回答

流量控制保护接收端，避免发送速度超过其缓冲和处理能力，主要信号是 `rwnd`；拥塞控制保护网络路径，避免在瓶颈链路制造持续排队和丢包，主要由发送端维护 `cwnd`。可发送在途量通常受 `min(rwnd, cwnd)` 约束。

深度解析：Know-How + Know-Why

- 接收端通过 TCP Window 字段通告可接收空间。
- 拥塞控制根据 ACK、loss、ECN、RTT 等信号估计网络容量。
- 接收端再强也无法绕过网络瓶颈；网络再空也不能把接收端缓冲打爆。

面试官常见追问

- Zero Window 是流控还是拥塞?
- ECN 属于哪一类信号?
- 应用读得慢会如何影响 rwnd?

高频易错点

- 把 flow control 和 congestion control 当同义词。
- 说最终窗口是 rwnd+cwnd。

动手验证 / 抓包实验

用接收端故意不 read 的程序观察 Window 逐步缩小甚至 Zero Window；与注入丢包导致 cwnd 下降对比。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

37. Slow Start 为什么叫“慢启动”，它的增长其实有多快？

难度：进阶　**频率：**★★★★☆　**标签：**Slow Start / ssthresh　**定位：**核心真题

30 秒标准回答

Slow Start 是相对“直接以大窗口灌满未知网络”而言谨慎。经典上 cwnd 随 ACK 在每个 RTT 近似指数增长，直到达到 ssthresh 或检测到拥塞，再进入更温和的拥塞避免阶段。

深度解析：Know-How + Know-Why

- 初始 cwnd 现代实现通常不是 1 MSS，具体遵循实现与标准演进。
- 指数增长意味着短连接性能高度受初始窗口和 RTT 影响。
- loss/ECN 会改变 ssthresh、cwnd，并进入相应恢复状态。

面试官常见追问

- 为什么短 Web 请求受 Slow Start 影响明显?
- ssthresh 如何更新?
- 拥塞窗口为何以字节/MSS 为单位变化?

高频易错点

- 机械背 1,2,4,8，不说明它是“每 RTT 近似翻倍”。
- 认为慢启动只在连接第一次建立时发生。

动手验证 / 抓包实验

在带宽足够、RTT 可控的 netem 环境抓取新连接，观察 `ss -ti` 中 cwnd 随时间增长。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

38. Fast Retransmit 与 Fast Recovery 分别是什么？

难度：进阶　**频率：**★★★★☆　**标签：**Reno / Loss Recovery　**定位：**核心真题

30 秒标准回答

Fast Retransmit 通过重复 ACK 等信号在 RTO 到期前重传疑似丢失段；Fast Recovery 则是在确认网络仍有数据流动后，避免像超时那样把发送速率降到极低，尽快恢复拥塞窗口。

深度解析：Know-How + Know-Why

- Reno 时代的经典规则是 3 duplicate ACK 触发快速重传。
- 超时通常代表更严重的拥塞/ACK 时钟中断，因此窗口缩减更激进。
- 现代 Linux 的 recovery 还会结合 SACK、PRR、RACK 等机制。

面试官常见追问

- 为什么 dupACK 表明网络可能仍能传包?
- PRR 解决了什么?
- NewReno 与 Reno 在多丢包恢复上有什么差别?

高频易错点

- 把 Fast Retransmit 和 Fast Recovery 合并成一个概念。
- 认为快速恢复“不降低 cwnd”。

动手验证 / 抓包实验

注入单包丢失与持续丢失，比较是否进入 fast retransmit 以及何时落到 RTO。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

39. rwnd、cwnd、ssthresh 三者是什么关系？

难度：进阶 **频率：**★★★★★ **标签：**窗口 / 拥塞状态 **定位：**核心真题

30 秒标准回答

`rwnd` 是接收端通告的接收能力；`cwnd` 是发送端根据网络拥塞状态维护的在途上限；`ssthresh` 是经典拥塞控制中 Slow Start 与 Congestion Avoidance 的切换参考。真正能飞行的数据受前两者最小值限制。

深度解析：Know-How + Know-Why

- rwnd 可受应用读取速度和 receive buffer 影响。
- cwnd 与 RTT、ACK、loss/ECN 等网络反馈相关。
- ssthresh 不是“网络带宽”，只是控制算法中的状态变量。

面试官常见追问

- 为什么高 BDP 链路需要 Window Scale?
- cwnd 很大但吞吐仍低可能是什么原因?
- rwnd 很大是否代表接收端应用处理很快?

高频易错点

- 把 ssthresh 当成第三个独立硬限制并与前两者取最小。
- 认为 cwnd 由接收端发给发送端。

动手验证 / 抓包实验

`ss -ti` 查看 cwnd/rtt；结合 `/proc/sys/net/ipv4/tcp_rmem` 和 socket buffer 配置理解 rwnd 来源。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

40. TCP Reno 与 CUBIC 的核心差异是什么？

难度：进阶 **频率：**★★★★☆ **标签：**Reno / CUBIC **定位：**核心真题

30 秒标准回答

Reno 在拥塞避免阶段近似线性增窗，适合早期较低 BDP 网络；CUBIC 以距离上次拥塞点的时间为变量使用三次函数增长，在高带宽、高 RTT 链路上更快恢复到先前窗口附近，同时保持一定稳定性。

深度解析：Know-How + Know-Why

- CUBIC 仍是 loss-based congestion control，丢包通常触发窗口降低。
- 其曲线在上次最大窗口附近增长较慢，远离该点时增长更积极。
- Linux 长期广泛使用 CUBIC，现代标准已将其规范化。

面试官常见追问

- CUBIC 为什么比 Reno 更适合长肥网络?
- CUBIC 如何与 Reno-friendly 行为兼容?
- 为什么 loss-based 容易造成 bufferbloat?

高频易错点

- 把 CUBIC 说成“按三次方每 RTT 增长”。
- 认为 CUBIC 不依赖丢包信号。

动手验证 / 抓包实验

查看 `sysctl net.ipv4.tcp_congestion_control`；在可控环境切换 reno/cubic 比较高 RTT 下吞吐。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

41. BBR 与 CUBIC 的核心思想差在哪里？

难度：进阶　**频率：**★★★★☆　**标签：**BBR / Model-based CC　**定位：**核心真题

30 秒标准回答

CUBIC 主要以丢包作为拥塞信号并调整窗口；BBR 更偏模型驱动，估计瓶颈带宽与最小传播 RTT，据此控制 pacing rate 和 in-flight 数据，目标是接近高吞吐、低排队，而不是等到丢包才认为拥塞。

深度解析：Know-How + Know-Why

- BBR 关注 delivery rate 与 RTT 样本，周期性探测带宽/RTprop。
- 模型估计误差、公平性和不同版本行为是 BBR 实际部署的重要议题。
- “不以丢包为主信号”不等于完全忽略丢包。

面试官常见追问

- BBR 为什么强调 pacing?
- BBR 与 bufferbloat 的关系?
- 不同拥塞算法竞争时公平性如何?

高频易错点

- 说 BBR “没有 cwnd”。
- 把 BBR 描述成“永不丢包”。

动手验证 / 抓包实验

若内核支持，切换 bbr 与 cubic，在高延迟+大 buffer 模拟链路对比 RTT 膨胀与吞吐。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

42. Nagle Algorithm 解决什么？为什么交互式 RPC 常设置 TCP_NODELAY？

难度：进阶　**频率：**★★★★☆　**标签：**Nagle / Tinygram　**定位：**核心真题

30 秒标准回答

Nagle 通过抑制大量小而未确认的 TCP Segment 来提高链路效率。对低延迟 RPC、游戏、交互式协议，小消息等待聚合可能增加尾延迟，因此常设置 `TCP_NODELAY` 禁用 Nagle，但这并不会让 TCP 变成“实时协议”。

深度解析：Know-How + Know-Why

- Nagle 与 delayed ACK 的组合在某些模式下会造成明显延迟。
- 禁用 Nagle 会增加小包数量和每包开销，需权衡 latency 与 packet rate。
- 应用层批处理有时比完全依赖 Nagle 更可控。

面试官常见追问

- Delayed ACK 是什么?
- 为什么一次 write 小于 MSS 不一定马上发包?
- RPC 框架为什么常自己做 batching?

高频易错点

- 认为 TCP_NODELAY 能消除所有 TCP 延迟。
- 认为 Nagle 只是“缓存 200ms”。

动手验证 / 抓包实验

写小消息 ping-pong，比较 TCP_NODELAY 开/关下 p50/p99 延迟和 packet count。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

43. TCP_NODELAY 到底改变什么，不改变什么？

难度：进阶 **频率：**★★★☆☆ **标签：**TCP_NODELAY / 延迟 **定位：**核心真题

30 秒标准回答

TCP_NODELAY 主要禁用 Nagle 对小包的聚合等待，使可发送的小数据更积极进入发送路径。它不改变路由 RTT、拥塞、RTO、丢包、调度、TLS、应用队列等延迟来源。

深度解析：Know-How + Know-Why

- 对大块顺序传输几乎没有“神奇提速”。
- 对高频 tiny write 可能提升时延但恶化包率与 CPU 效率。
- 合理做法是基于协议消息模式和延迟预算测量，而非默认全开/全关。

面试官常见追问

- TCP_CORK 与 TCP_NODELAY 有何区别?

- 为什么游戏协议可能仍不用 TCP?
- `writev` 能怎样减少小 write?

高频易错点

- 把该选项称为“禁用 TCP 缓冲”。
- 认为设置后 send() 就立即物理发出。

动手验证 / 抓包实验

用 perf/tcpdump 比较大量 20B write 在 NODELAY 开关下的 segments/s 与 CPU。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

44. BDP 是什么？为什么高带宽长 RTT 链路需要大窗口？

难度：进阶 **频率：**★★★★★ **标签：**BDP / Long Fat Network **定位：**核心真题

30 秒标准回答

BDP=Bandwidth×RTT，表示要让链路持续满载，理论上需要约这么多数据同时处于 in-flight。若 TCP 的 `min(cwnd,rwnd)` 远小于 BDP，即使链路带宽很高，也会因每个 RTT 可发送的数据有限而跑不满。

深度解析：Know-How + Know-Why

- 1Gbps×100ms≈12.5MB 的在途数据量。
- Window Scale、socket buffers、拥塞控制都可能成为上限。
- “带宽大但下载慢”需要同时看 RTT、窗口、loss 和应用并发。

面试官常见追问

- 为什么卫星链路 BDP 特别大?
- 并行多连接为什么有时能绕过单连接窗口限制?
- QUIC 也需要考虑 BDP 吗?

高频易错点

- 只看 Mbps，不看 RTT。
- 认为增大 socket buffer 必然提升吞吐。

动手验证 / 抓包实验

用 `tc netem delay` 模拟 100ms RTT，配合 `iperf3` 与不同 socket buffer/cwnd 观察单连接吞吐。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

45. TCP 半连接队列、全连接队列与 listen(backlog) 有什么关系？

难度：工程　**频率：**★★★★★　**标签：**SYN Queue / Accept Queue　**定位：**核心真题

30 秒标准回答

监听 socket 收到 SYN 后进入半连接状态并等待握手完成；完成握手后连接进入 accept queue，等待应用 `accept()` 取走。Linux 中两类队列受不同内核参数与 backlog 上限影响，排障时必须区分“握手没完成”与“握手完成但应用没 accept”。

深度解析：Know-How + Know-Why

- `tcp_max_syn_backlog` 影响 SYN_RECV 容量；`somaxconn` 限制 listen backlog 的上界。
- accept queue 满可能导致新完成连接被丢弃/延迟，表现为客户端 connect 异常或重传。
- SYN Cookie 主要针对半连接压力，不解决应用 accept 太慢。

面试官常见追问

- 如何判断到底哪个队列满?
- 为什么 worker starvation 会导致 accept queue 堆积?
- SO_REUSEPORT 能怎样分摊监听压力?

高频易错点

- 把 backlog 只解释成“最大并发连接数”。
- 认为调大 somaxconn 就能解决任何连接失败。

动手验证 / 抓包实验

查看 `ss -lnt` 的 Recv-Q/Send-Q、内核 counters，并用小 backlog 服务做受控压测。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

第五章 UDP、QUIC 与 HTTP/3 传输基础

本章目标

从“无连接数据报”到现代用户态可靠传输协议。

建议：先能口述结论，再能画出关键时序/状态，最后通过命令或抓包验证。

46. 如果基于 UDP 实现可靠传输，需要补哪些机制？

难度：进阶	频率：★★★★☆	标签：Reliable UDP / Protocol Design	定位：核心真题

30 秒标准回答

至少要补连接/会话标识、序号、ACK、超时重传、重复检测、乱序重排、流量控制、拥塞控制、分片/重组、路径 MTU、加密认证和故障恢复等。所谓“可靠 UDP”最终会逐渐长成一个完整传输协议。

深度解析：Know-How + Know-Why

- 可靠性与拥塞控制是两个不同维度，不能只加 ACK。
- 报文边界是 UDP 的优势，但超大 datagram 会遇到 IP fragmentation 风险。
- 协议还要处理 NAT rebinding、路径变化、DoS 防护等现实问题。

面试官常见追问

- 为什么自己写 RUDP 很容易出错?
- 什么场景可能仍值得定制 UDP 协议?
- QUIC 解决了其中哪些问题?

高频易错点

- 只说“加一个重传”。
- 忽略拥塞控制对互联网公平性的责任。

动手验证 / 抓包实验

设计一个简化 RUDP 状态机文档：编号、ACK、RTO、窗口、重排、连接关闭；不建议在公网生产自行实现。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

47. UDP 为什么不存在 TCP 那种“粘包”语义？

难度：基础	频率：★★★★☆	标签：Datagram / Message Boundary	定位：核心真题

30 秒标准回答

UDP 保留 datagram 边界：一次 UDP send 形成一个数据报，接收 API 以数据报为单位取消息。TCP 则只保证字节顺序，没有应用消息边界。需要注意 UDP 接收缓冲太小会截断数据报，而不是自动拆成下一次 recv。

深度解析：Know-How + Know-Why

- 多个 UDP datagram 不会被内核自动拼成一个应用消息。
- 一个超大 UDP datagram 可能在 IP 层分片，丢一个 fragment 就导致整个 datagram 丢失。
- 应用仍需定义请求 ID、重试和去重等语义。

面试官常见追问

- UDP recv buffer 小于 datagram 会怎样?
- 为什么 DNS 通常控制 UDP response 大小?
- QUIC packet 与 UDP datagram 的关系?

高频易错点

- 认为 UDP “绝对不会分片”。
- 把 UDP message boundary 等同于可靠消息。

动手验证 / 抓包实验

发送不同长度 UDP datagram，用不同 recv buffer 观察截断与边界。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

48. QUIC 为什么建立在 UDP 上，而不是直接改 TCP？

难度：进阶	频率：★★★★★	标签：QUIC / Deployability	定位：核心真题

30 秒标准回答

UDP 提供最小可部署的 IP 之上承载，让 QUIC 可在用户态快速演进，同时避免中间设备对新 TCP 扩展的 ossification。QUIC 自己实现可靠流、拥塞控制、loss recovery、TLS 1.3、安全连接 ID 与迁移等能力。

深度解析：Know-How + Know-Why

- QUIC 不是“UDP+一点重传”，而是完整加密传输协议。
- 用户态实现能更快迭代，而不用等待操作系统 TCP 栈升级。
- UDP 的广泛中间设备支持仍会遇到阻断/限速，实际部署要做 fallback。

面试官常见追问

- 什么是 protocol ossification?
- QUIC 为什么几乎所有 header/metadata 都尽量加密?
- UDP 被企业网络封锁时怎么办?

高频易错点

- 说 QUIC 选择 UDP 只是因为“快”。
- 认为 QUIC 不需要拥塞控制。

动手验证 / 抓包实验

用 `curl --http3`（环境支持时）抓 QUIC 流量，对比 TCP/TLS 的握手包。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

49. HTTP/2 已有多路复用，为什么 HTTP/3 仍需要 QUIC？

难度：进阶 **频率：**★★★★★ **标签：**HTTP/2 / HTTP/3 / HOL **定位：**核心真题

30 秒标准回答

HTTP/2 在应用层把请求分成独立 stream，但所有 frame 最终仍进入一个有序 TCP 字节流。底层 TCP 某个 Segment 丢失会阻塞后续字节交付，所有 HTTP/2 stream 都可能受影响。QUIC 在传输层提供独立 stream，某流丢包通常不阻塞其他流。

深度解析：Know-How + Know-Why

- HTTP/2 解决的是 HTTP/1.x 的许多应用层队头阻塞，但没有消除 TCP transport HOL。
- QUIC 的 packet number 与 stream offset 分离，使不同 stream 可独立重组。
- HTTP/3 还引入 QPACK 适应乱序、多流传输。

面试官常见追问

- 为什么 QUIC 自身的 crypto stream 仍需要有序?
- HTTP/3 是否完全没有任何 HOL?
- 一条 QUIC connection 里拥塞控制是否仍共享?

高频易错点

- 简单说“HTTP/2 没有多路复用”。
- 说 HTTP/3 可以让一个物理包里的所有内容独立丢失恢复。

动手验证 / 抓包实验

在高 RTT+丢包模拟环境对比 HTTP/2 与 HTTP/3 多资源并行请求的 tail latency。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

50. QUIC Connection Migration 为什么适合移动网络？

难度：进阶	频率：★★★★☆	标签：Connection ID / Path Validation	定位：核心真题

30 秒标准回答

TCP 连接强依赖 IP/Port 四元组，手机从 Wi-Fi 切到 5G 时源地址变化常导致连接失效。QUIC 用 Connection ID 把逻辑连接与单一路径解耦，并通过 PATH_CHALLENGE/PATH_RESPONSE 等机制验证新路径，从而支持 NAT rebinding 与网络切换。

深度解析：Know-How + Know-Why

- 迁移不意味着无条件信任任意新源地址，需要 path validation 防止放大攻击。
- 拥塞控制状态在新路径上的继承需要谨慎，因为新路径容量可能不同。
- 服务器也可主动发行多个 Connection ID 以降低可关联性。

面试官常见追问

- TCP MPTCP 与 QUIC migration 有何区别?
- QUIC 如何防止迁移被攻击者劫持?
- 连接迁移对负载均衡有什么挑战?

高频易错点

- 认为 Connection ID 是固定明文用户 ID。
- 认为路径切换后拥塞窗口一定原样继承。

动手验证 / 抓包实验

在支持 QUIC 的测试程序中切换本机网络接口，观察连接是否保持及路径验证事件。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

51. QUIC 为什么能减少首次/恢复连接延迟？0-RTT 的收益与风险是什么？

难度：进阶　**频率：**★★★★☆　**标签：**QUIC Handshake / 0-RTT　**定位：**核心真题

30 秒标准回答

QUIC 将传输握手与 TLS 1.3 紧密集成，首次连接可在较少往返内建立加密会话；已有会话状态时可发送 0-RTT early data，减少等待。但 0-RTT 存在 replay 风险，不适合不可幂等的敏感操作。

深度解析：Know-How + Know-Why

- 首次握手仍受 DNS、网络 RTT、证书验证等影响，不是“0 RTT 建连”。
- 0-RTT data 在安全属性上弱于握手完成后的 1-RTT 数据。
- 服务端需要 anti-replay 策略，应用层也应限制 early data 的语义。

面试官常见追问

- 为什么 GET 静态资源更适合 0-RTT?
- TLS 1.3 session ticket 在恢复里起什么作用?
- QUIC Retry 什么时候会增加 RTT?

高频易错点

- 说 QUIC 所有连接都是 0-RTT。
- 忽略 replay 对支付/下单等操作的风险。

动手验证 / 抓包实验

对支持 HTTP/3 的站点执行首次与复用连接请求，结合 qlog/Wireshark 比较握手往返。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

52. QUIC 使用 UDP，是否就不需要拥塞控制和流量控制？

难度：基础　**频率：**★★★★☆　**标签：**QUIC Flow/Congestion Control　**定位：**核心真题

30 秒标准回答

完全错误。UDP 基础协议不提供拥塞与流控，QUIC 必须自己实现连接级拥塞控制、连接级与流级 flow control、ACK/loss recovery。否则一个 QUIC sender 可以轻易压垮网络或接收端。

深度解析：Know-How + Know-Why

- QUIC 的 MAX_DATA、MAX_STREAM_DATA 等 frame 用于 flow control。
- 拥塞控制通常作用于整条 connection 的路径容量，而不是每个 stream 各自无限发送。
- ACK frame、packet number 与 loss detection 共同驱动恢复与拥塞控制。

面试官常见追问

- HTTP/3 stream flow control 与 QUIC connection flow control 如何叠加?
- QUIC ACK 为什么不使用 TCP 那种字节 ACK?
- QUIC 可以使用 CUBIC/BBR 吗?

高频易错点

- 把 UDP 的“无拥塞控制”误传递给 QUIC。
- 认为每个 QUIC stream 都有完全独立的带宽。

动手验证 / 抓包实验

阅读一个 QUIC qlog，定位 ACK、MAX_DATA、loss、cwnd 事件并画出时间线。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

第六章 DNS、全局调度与 CDN

本章目标

掌握域名解析、缓存、一致性与边缘加速。

建议：先能口述结论，再能画出关键时序/状态，最后通过命令或抓包验证。

53. 输入域名后，DNS 完整解析链路是什么？

难度：基础 **频率：**★★★★★ **标签：**DNS / Resolver / Cache **定位：**核心真题

30 秒标准回答

应用先检查浏览器/系统等本地缓存与 hosts，随后通常把查询交给递归 Resolver。若缓存未命中，Resolver 按 Root→TLD→Authoritative 的委派链获取最终记录并按 TTL 缓存。真实链路还可能包含 CNAME、多级 DNS、DoH/DoT 与负载调度。

核心时序 / 心智模型

```
Client
  ↓
Recursive Resolver
  ↓
Root → TLD → Authoritative
  ↓
Answer + TTL
  ↓
Cache
```

深度解析：Know-How + Know-Why

- Root 不会返回所有域名 IP，而是告诉 Resolver 去问相应 TLD。
- TLD 再返回目标域的权威服务器信息。
- Resolver 的缓存使绝大多数查询无需每次走完整根链路。

面试官常见追问

- CNAME 链过长会造成什么?
- 为什么 DNS 修改不是瞬间生效?
- 浏览器和 OS 的缓存策略是否完全遵循同一个 TTL?

高频易错点

- 说浏览器直接依次请求 Root/TLD。
- 忽略递归 Resolver 缓存。

动手验证 / 抓包实验

使用 `dig +trace` 观察委派链，再用普通 `dig` 对比递归 Resolver 的结果与 TTL。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

54. DNS 递归查询与迭代查询有什么区别？

难度：基础 **频率：**★★★★☆ **标签：**Recursive / Iterative **定位：**核心真题

30 秒标准回答

递归查询要求被询问者给出最终答案或失败；迭代查询允许服务器返回“我不知道最终答案，但你下一步去问这里”。典型互联网 DNS 中，终端向递归 Resolver 发递归请求，Resolver 再以迭代方式查询 Root/TLD/Authoritative。

深度解析：Know-How + Know-Why

- 递归 Resolver 代替客户端承担多次查询与缓存。
- 权威服务器通常只回答自己负责区域的数据，不帮你递归整个 DNS 树。
- 开放递归 Resolver 如果配置不当会被滥用于反射放大攻击。

面试官常见追问

- 什么是 DNS Referral?
- 为什么企业常部署本地递归 Resolver?
- Authoritative 与 Recursive 能否由同一软件提供?

高频易错点

- 把“递归/迭代”与“UDP/TCP”混为一谈。
- 认为所有 DNS 服务器都会帮客户端递归。

动手验证 / 抓包实验

用 `dig +norecurse` 查看某些服务器返回的 referral/权威行为。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

55. 为什么“DNS 使用 UDP”是错误或不完整答案？

难度：基础 **频率：**★★★★★ **标签：**DNS / UDP / TCP **定位：**核心真题

30 秒标准回答

DNS 同时使用 UDP 与 TCP。普通查询常优先 UDP；当响应被截断、使用较大 DNSSEC 数据、进行 zone transfer 等场景会使用 TCP。现代 DNS 还可运行在 TLS/HTTPS/QUIC 等安全传输上。

深度解析：Know-How + Know-Why

- 传统 UDP/TCP DNS 都通常使用 53 端口。
- EDNS 扩大 UDP 可承载的 DNS 消息，但仍需考虑路径 MTU 与放大攻击。
- DoT/DoH 改变的是传输与隐私模型，不改变 DNS 名称层次本身。

面试官常见追问

- TC bit 表示什么?
- AXFR 为什么使用 TCP?
- DoH 与 DNSSEC 解决的是同一问题吗?

高频易错点

- 回答“DNS 永远 UDP 53”。
- 认为用了 DoH 就自动防 DNS 缓存污染。

动手验证 / 抓包实验

用 `dig +tcp` 与普通 `dig` 比较；选择 DNSSEC 大响应查看 UDP size 与可能的 fallback。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

56. A、AAAA、CNAME、MX、NS、TXT 分别表达什么？

难度：基础 **频率：**★★★★☆ **标签：**DNS Records **定位：**核心真题

30 秒标准回答

A 把名称映射到 IPv4，AAAA 映射到 IPv6；CNAME 表示别名指向 canonical name；MX 指定邮件交换服务器；NS 声明区域权威 DNS；TXT 携带文本型数据，常用于 SPF、DKIM/验证等。

深度解析：Know-How + Know-Why

- CNAME 的目标仍需继续解析 A/AAAA。
- MX 记录包含 preference，且目标应是主机名而非任意 URL。
- TXT 被广泛滥用为“可扩展元数据”载体，单条长度与分片需遵守 DNS 格式。

面试官常见追问

- 为什么 zone apex 传统上不宜直接做 CNAME?
- SRV 记录解决什么?
- CAA 与证书签发有什么关系?

高频易错点

- 把 CNAME 当作 HTTP 重定向。
- 认为 MX 直接存邮件服务器 IP。

动手验证 / 抓包实验

用 `dig A/AAAA/CNAME/MX/NS/TXT` 查询一个真实域名，画出记录依赖关系。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

57. DNS TTL 与 Negative Caching 是什么？

难度：进阶 **频率：**★★★★☆ **标签：**TTL / NXDOMAIN **定位：**核心真题

30 秒标准回答

TTL 指定记录可被缓存的时间，降低权威 DNS 压力和查询延迟；负缓存则把 NXDOMAIN/NODATA 等“没有记录”的结果也缓存一段时间，避免持续重复查询不存在的名称。

深度解析：Know-How + Know-Why

- TTL 越长，缓存命中更高但变更收敛更慢。
- 上线迁移常提前降低 TTL，而不是修改记录后才降低。
- 负缓存过长会让刚创建的域名在部分用户处继续“查不到”。

面试官常见追问

- 为什么 TTL=0 也不一定全球立即一致?
- NXDOMAIN 与 SERVFAIL 能否同样缓存?
- CDN 为什么会精细控制 DNS TTL?

高频易错点

- 认为只有成功答案能缓存。
- 把 DNS TTL 与 IP 包 TTL 混淆。

动手验证 / 抓包实验

先查询一个不存在的测试子域，再短时间重复查询，观察 Resolver 负缓存行为与 SOA 信息。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

58. DNS 如何做负载均衡？它的局限是什么？

难度：进阶 **频率：**★★★★☆ **标签：**DNS LB / GSLB **定位：**核心真题

30 秒标准回答

DNS 可返回多个 A/AAAA，或基于地域、网络、健康状态、权重动态返回不同地址，实现粗粒度全局流量调度。局限是 Resolver 缓存、TTL、客户端未必按预期轮询，以及 Resolver 位置并不总等于用户位置。

深度解析：Know-How + Know-Why

- DNS 适合做全局入口调度，不擅长每请求级精确负载均衡。
- 健康切换速度受 TTL 与缓存行为影响。
- EDNS Client Subnet 能改善地理判断，但带来隐私与缓存碎片问题。

面试官常见追问

- DNS LB 与 L4/L7 LB 如何配合?
- 为什么健康检查正常但部分用户仍访问旧 IP?
- Anycast 与 GeoDNS 如何选择?

高频易错点

- 认为返回多个 IP 就会严格平均流量。
- 忽略客户端/Resolver 缓存导致的长尾。

动手验证 / 抓包实验

配置一个测试域返回两个地址，用不同 resolver 与 TTL 观察缓存、顺序和客户端选择。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

59. DNS Cache Poisoning 是什么？DNSSEC、DoH/DoT 分别保护什么？

难度：进阶 **频率：**★★★★☆ **标签：**DNS Security **定位：**核心真题

30 秒标准回答

缓存投毒试图把伪造 DNS 数据写入 Resolver，使域名解析到攻击者地址。DNSSEC 通过签名链验证 DNS 数据来源与完整性；DoH/DoT 主要加密客户端到 Resolver 的传输，防止链路窃听/篡改，但本身不证明上游 DNS 数据真实。

深度解析：Know-How + Know-Why

- 随机 transaction ID 与源端口提升伪造响应难度。
- DNSSEC 依赖 DS/DNSKEY/RRSIG 建立从根开始的信任链。
- DoH 可能把 DNS 观察点集中到少数提供商，需要单独评估隐私模型。

面试官常见追问

- DoH 是否能防恶意 Resolver?
- DNSSEC 能隐藏用户查询内容吗?
- 什么是 DNS amplification attack?

高频易错点

- 把 DoH 当作 DNSSEC 的替代。
- 认为 HTTPS 页面就不需要可信 DNS。

动手验证 / 抓包实验

用 `dig +dnssec` 查看带 RRSIG 的记录；比较普通 DNS 与 DoH 的抓包可见性。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

60. CDN 为什么能加速？缓存命中率为什么只是其中一部分？

难度：进阶 **频率：**★★★★★ **标签：**CDN / Edge / Origin **定位：**核心真题

30 秒标准回答

CDN 通过分布式边缘节点把内容与连接终止点靠近用户，减少 RTT、跨洲链路与源站压力。除缓存命中外，还通过 Anycast/DNS 调度、TLS session reuse、HTTP/2/3、压缩、连接复用、Origin Shield 等优化动态和静态流量。

深度解析：Know-How + Know-Why

- Cache Hit 可直接从 edge 返回；Miss 仍可复用 edge→origin 长连接和优化路由。
- Origin Shield 让多个 edge miss 汇聚到更少的上游请求，降低源站惊群。
- 动态 API 也可能从就近 TLS termination、连接池和全局网络中获益。

面试官常见追问

- CDN Cache Key 如何设计?
- 为什么 Cookie/Query 参数会破坏命中率?
- 如何避免缓存用户私有数据?

高频易错点

- 把 CDN 简化成“静态文件复制”。
- 只追求 hit ratio，忽略错误 cache key 的数据泄漏风险。

动手验证 / 抓包实验

对同一资源用 `curl -I` 多次请求，观察 Age、Via/X-Cache、Server-Timing 等缓存相关头。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

第七章 HTTP/1.1、HTTP/2 与应用层协议

本章目标

从 HTTP 语义深入到 framing、缓存、多路复用与 RPC。

建议：先能口述结论，再能画出关键时序/状态，最后通过命令或抓包验证。

61. HTTP/1.1 Request/Response 报文结构是什么？

难度：基础　频率：★★★★★　标签：HTTP Message / Framing　定位：核心真题

30 秒标准回答

HTTP/1.1 消息由 start-line、header fields、空行和可选 body 构成。请求起始行包含 method、target、version；响应起始行包含 version、status code、reason phrase。真正难点是 message body 的长度与连接复用边界。

深度解析：Know-How + Know-Why

- Host 在 HTTP/1.1 虚拟主机中至关重要。
- Content-Length、Transfer-Encoding、状态码和方法共同决定是否有 body。
- Header 是大小写不敏感的字段名语义，但具体值规则由字段定义。

面试官常见追问

- HEAD 响应为什么通常没有 body?
- 204/304 的 body 规则是什么?
- HTTP request target 有哪些形式?

高频易错点

- 认为空行之后一定有 body。
- 只背 GET 行格式，不理解 framing。

动手验证 / 抓包实验

用 `nc` 手工向本地 HTTP server 发送原始 HTTP/1.1 请求，观察缺少 Host/Content-Length 时行为。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

62. GET 与 POST 真正的区别是什么？safe、idempotent、cacheable 如何区分？

难度：基础　频率：★★★★★　标签：HTTP Semantics　定位：核心真题

30 秒标准回答

GET 的语义是获取资源表示，通常是 safe 且 idempotent；POST 表示让目标资源处理请求内容，通常不保证幂等。URL 参数与 body 只是常见约定，不是安全性边界。safe 指不期望改变服务器状态；idempotent 指重复执行效果与一次相同。

深度解析：Know-How + Know-Why

- PUT、DELETE 通常定义为 idempotent，但不一定 safe。
- POST 可以被缓存，只是需要明确缓存语义，不能说“永远不能缓存”。
- 敏感参数放 POST body 仍会被浏览器、代理、日志、服务端看到，HTTPS 才提供传输机密性。

面试官常见追问

- 支付接口为什么要设计幂等键?
- DELETE 重复调用为什么仍算 idempotent?
- GET 能不能有 body?

高频易错点

- “GET 长度有限、POST 无限”是错误概括。
- 把 method 名字当成权限控制。

动手验证 / 抓包实验

为“创建订单”“查询订单”“更新全部字段”“部分更新”设计 HTTP method，并说明幂等策略。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

63. 常见 HTTP 状态码中，301/302/307/308、401/403、502/504 怎么区分？

难度：基础　频率：★★★★★　标签：Status Code / Gateway　定位：核心真题

30 秒标准回答

301/308 表示永久重定向，302/307 表示临时；307/308 明确保留原 method/body。401 通常表示缺少/无效认证并可带 WWW-Authenticate，403 表示服务器理解但拒绝授权。502 是 gateway 从 upstream 得到无效响应，504 是等待 upstream 超时。

深度解析：Know-How + Know-Why

- 3xx 的客户端行为历史兼容性很复杂，307/308 更适合要求 method 不变。
- 429 通常用于速率限制，503 用于暂时不可用。
- 应用框架返回 500 与反向代理返回 502/504 的定位层不同。

面试官常见追问

- 为什么登录失效通常是 401 而非 403?
- 502 可能由 connection reset 触发吗?
- 504 与客户端 408 有什么不同?

高频易错点

- 把 401 解释成“未授权”而 403 “未认证”，方向常说反。
- 认为所有 5xx 都是后端应用 Bug。

动手验证 / 抓包实验

构造 upstream sleep、upstream reset、认证失败三种场景，观察 Nginx/Envoy 返回的状态码和日志。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

64. HTTP “无状态”是什么意思？Cookie、 Session、 Token 为什么出现？

难度：基础 **频率：**★★★★☆ **标签：**Stateless / Session **定位：**核心真题

30 秒标准回答

HTTP 协议本身不要求服务器依赖某条连接中隐式保存的前序请求状态；每个请求携带足够上下文。业务登录态需要额外机制：Cookie 让客户端携带标识，Session 把状态放服务器，Token 可把部分声明放到客户端携带的签名数据中。

深度解析：Know-How + Know-Why

- Cookie 是浏览器存储与自动发送机制，不等于 Session。
- Session ID 通常是随机不透明标识，真正用户状态在服务端存储。
- JWT 是 token 格式之一，不等于“无状态就一定更安全”。

面试官常见追问

- Cookie 的 Secure/HttpOnly/SameSite 分别做什么?
- Session 水平扩展怎么做?
- JWT 撤销为什么困难?

高频易错点

- 说 HTTP 无状态意味着“服务器不能存数据库”。
- 把 Cookie 值直接等同于用户密码。

动手验证 / 抓包实验

抓浏览器请求，观察登录前后 Set-Cookie/Cookie；说明为什么 HttpOnly Cookie 仍会自动随请求发送。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

65. HTTP Persistent Connection 与 TCP 长连接是什么关系？

难度：基础	频率：★★★★☆	标签：Keep-Alive / Connection Reuse	定位：核心真题

30 秒标准回答

HTTP persistent connection 是“多个 HTTP 事务复用底层传输连接”的应用层策略；TCP 只提供字节流，不知道哪些字节属于哪个 HTTP 请求。HTTP/1.1 默认倾向持久连接，HTTP/2/3 更进一步依赖长连接多路复用。

深度解析：Know-How + Know-Why

- 持久连接减少 TCP/TLS 握手成本和 Slow Start。
- 连接太长也会受负载均衡、服务器 drain、NAT timeout 等影响。
- HTTP Keep-Alive 与 TCP Keepalive 是两个完全不同概念。

面试官常见追问

- TCP Keepalive 与 HTTP Keep-Alive 有何区别?
- 为什么连接池需要 max idle time?
- HTTP/2 是否还需要多个 TCP 连接?

高频易错点

- 把两种 Keepalive 混淆。
- 认为 persistent connection 永远不会被服务端关闭。

动手验证 / 抓包实验

用 `curl -v`/连接池客户端观察同一 host 多请求的 connection reuse。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

66. Content-Length 与 chunked transfer encoding 解决什么？

难度：进阶 **频率：**★★★★☆ **标签：**HTTP/1.1 Framing **定位：**核心真题

30 秒标准回答

HTTP/1.1 需要明确一个 message body 在持久连接中何时结束。已知长度可用 Content-Length；生成时未知长度可使用 chunked transfer coding，以十六进制 chunk size 分段，最终以 0 chunk 结束。二者错误组合会产生严重解析歧义。

深度解析：Know-How + Know-Why

- chunked 是 HTTP/1.1 的消息传输编码，不是 TCP 分片机制。
- HTTP/2/3 使用 frame 自带长度，不再使用 HTTP/1.1 的 chunked framing。
- 代理对 Content-Length/Transfer-Encoding 解释不一致会导致 Request Smuggling。

面试官常见追问

- 什么是 CL.TE / TE.CL smuggling?
- 流式响应为什么不一定需要 chunked 在 HTTP/2 中?
- Trailer headers 有什么用途?

高频易错点

- 把 chunked 当成“压缩”。
- 认为每个 chunk 对应一个 TCP packet。

动手验证 / 抓包实验

用一个 streaming HTTP/1.1 server 输出 chunked response，抓取原始字节并手工解析 chunk size。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

67. Cache-Control 中 max-age、no-cache、no-store、private、must-revalidate 怎么理解？

难度：进阶 **频率：**★★★★★ **标签：**HTTP Cache **定位：**核心真题

30 秒标准回答

`max-age` 定义 freshness lifetime；`no-cache` 通常允许存储但复用前必须 revalidate；`no-store` 要求不存储；`private` 限制共享缓存存储；`must-revalidate` 要求过期后在未成功验证时不能随意使用 stale response。

深度解析：Know-How + Know-Why

- 缓存分为 browser/private cache 与 CDN/shared cache。
- 缓存策略需同时考虑 Vary、Authorization、Cookie 与 Cache Key。
- “强缓存/协商缓存”是常见教学术语，但最好映射到 freshness 与 validation 两个标准概念。

面试官常见追问

- s-maxage 与 max-age 有何区别?
- stale-while-revalidate 为什么能降尾延迟?
- Vary: Accept-Encoding 如何影响缓存键?

高频易错点

- 把 no-cache 解释成绝对不缓存。
- 忽略 private data 被共享 CDN 缓存的安全风险。

动手验证 / 抓包实验

对静态资源设置不同 Cache-Control，用浏览器 DevTools 观察 memory/disk cache、304 与网络请求。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

68. ETag/If-None-Match 与 Last-Modified/If-Modified-Since 如何工作？

难度：进阶　**频率：**★★★★☆　**标签：**Conditional Request / 304　**定位：**核心真题

30 秒标准回答

它们用于缓存验证：客户端持有旧 representation，发送 validator；服务器若判断资源未变化，可返回 304 不传 body。ETag 可表达更精确的版本标识，Last-Modified 基于时间且粒度/时钟可能有限。

深度解析：Know-How + Know-Why

- 强 ETag 要求字节级等价；弱 ETag 表示语义等价，使用 `W/` 前缀。
- If-None-Match 在常见 GET 缓存验证中优先于日期 validator。
- 304 响应仍包含必要的缓存元数据更新。

面试官常见追问

- 为什么静态文件可用 hash 做 ETag?
- 分布式服务器生成 ETag 有什么坑?
- If-Match 如何用于乐观并发控制?

高频易错点

- 认为 304 是“服务器没有响应”。
- 把 Last-Modified 当成可靠唯一版本号。

动手验证 / 抓包实验

用 `curl -I` 记录 ETag，再带 `If-None-Match` 请求验证 304；修改文件后再次观察。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

69. HTTP/1.1 与 HTTP/2 的核心差异是什么？

难度：进阶 **频率：**★★★★★ **标签：**HTTP/2 / Binary Framing **定位：**核心真题

30 秒标准回答

HTTP/2 保留 HTTP 方法、状态码和语义，但把传输表示改成二进制 frame，引入 stream、多路复用、HPACK 头压缩和流量控制，从而避免 HTTP/1.1 多连接/队列带来的大量效率问题。

深度解析：Know-How + Know-Why

- 一个 HTTP/2 connection 中有多个独立 stream，frame 可交错传输。
- Header block 通过 HPACK 压缩，减少重复 Cookie/User-Agent 等开销。
- HTTP/2 仍通常运行在单个 TCP connection 上，因此保留 transport HOL。

面试官常见追问

- HTTP/2 Server Push 为什么实际使用减少?
- stream ID 奇偶规则是什么?
- HTTP/2 flow control 与 TCP flow control 是什么关系?

高频易错点

- 认为 HTTP/2 改变了 GET/POST 语义。
- 说 HTTP/2 完全没有队头阻塞。

动手验证 / 抓包实验

使用 `curl --http2 -v` 和浏览器 DevTools，观察同一 connection 的多 stream 请求。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

70. HTTP/2 为什么仍存在 Head-of-Line Blocking？

难度：进阶 **频率：**★★★★★ **标签：**HOL / TCP **定位：**核心真题

30 秒标准回答

HTTP/2 的 stream 在应用 framing 层独立，但所有 frame 字节仍必须按 TCP 序列顺序交付。TCP 中一个 Segment 丢失形成缺口时，即使后续包含的是其他 HTTP stream 的 frame，内核也不能越过缺口交给应用，因此多流一起等待。

深度解析：Know-How + Know-Why

- 这叫 transport-level HOL，与 HTTP/1.1 pipelining 的 application HOL 区别开。
- TCP SACK 可快速补丢包，但无法改变“按序字节流交付”的语义。
- QUIC 让不同 stream 各自按 offset 重组，减少跨流阻塞。

面试官常见追问

- HTTP/3 是否彻底消除所有 HOL?
- 一个 QUIC packet 包含多个 stream frame 丢失时会怎样?
- 为什么单 stream 内仍要求有序语义?

高频易错点

- 说 HTTP/2 multiplexing 就已经解决所有 HOL。
- 把“丢一个 TCP 包”理解成整个连接永久停止。

动手验证 / 抓包实验

在 HTTP/2 多资源加载场景注入 1% 丢包，观察所有 stream 的延迟抖动。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

71. HPACK 与 QPACK 为什么不能完全相同？

难度：进阶 **频率：**★★★☆☆ **标签：**Header Compression / **定位：**核心真题

QPACK

30 秒标准回答

HPACK 依赖 HTTP/2 可靠有序的 TCP 字节流；HTTP/3 运行在可独立乱序交付的 QUIC streams 上，如果直接复制 HPACK 的动态表依赖，就会重新制造跨流阻塞。QPACK 通过独立 encoder/decoder stream 与可控 blocking 机制适配乱序环境。

深度解析：Know-How + Know-Why

- 静态表减少常见 header 编码成本。
- 动态表能提升压缩率，但需要同步状态。
- QPACK 允许限制 blocked streams，在压缩率与 HOL 风险之间权衡。

面试官常见追问

- 为什么 header compression 可能引入安全侧信道?
- HPACK Huffman coding 是什么?
- QPACK encoder stream 丢包会怎样?

高频易错点

- 认为 QPACK 只是“更高压缩率的 HPACK”。
- 忽略动态表同步依赖。

动手验证 / 抓包实验

用 Wireshark 观察 HTTP/2 HEADERS/CONTINUATION 与 HTTP/3 QPACK streams 的不同表示。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

72. WebSocket、SSE、Long Polling 应该怎么选？

难度：进阶　**频率：**★★★★☆　**标签：**Realtime / Streaming　**定位：**核心真题

30 秒标准回答

Long Polling 仍是一问一答，只是服务端延迟响应；SSE 在 HTTP 上提供 server→client 的文本事件流，浏览器原生支持自动重连；WebSocket 建立双向全双工消息通道。选择依据是通信方向、消息频率、代理兼容、重连和运维复杂度。

深度解析：Know-How + Know-Why

- AI token streaming 常可用 SSE，简单、可穿透普通 HTTP 基础设施。
- 在线协作/聊天的频繁双向消息更适合 WebSocket。
- WebSocket 建连后仍需自己设计应用消息协议、心跳、认证刷新与 backpressure。

面试官常见追问

- SSE 为什么通常基于 UTF-8 文本?
- WebSocket 如何穿过 L7 proxy?
- HTTP/2/3 streaming 与 SSE 有何关系?

高频易错点

- 把 SSE 说成“单向 WebSocket”。
- 认为 WebSocket 天然保证业务消息不丢。

动手验证 / 抓包实验

分别实现 SSE token stream 与 WebSocket echo，对比浏览器 API、重连与代理配置。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

73. gRPC 为什么常运行在 HTTP/2 上？

难度：进阶 **频率：**★★★★☆ **标签：**gRPC / RPC / HTTP2 **定位：**核心真题

30 秒标准回答

gRPC 利用 HTTP/2 stream、多路复用、二进制 framing、流控与长连接承载 RPC，再叠加 protobuf message framing、service/method 语义、metadata、deadline、status 与多种 streaming 模式。它不是“HTTP/2 的另一个名字”。

深度解析：Know-How + Know-Why

- Unary、server streaming、client streaming、bidirectional streaming 都映射到 HTTP/2 stream。
- deadline/cancellation 应传播到下游，否则会制造无效工作。
- gRPC-Web 解决浏览器不能直接完整使用原生 gRPC transport 的限制。

面试官常见追问

- gRPC 与 REST 的主要 trade-off?
- 为什么 protobuf message 还需要 gRPC framing?
- HTTP/3 上能否跑 gRPC?

高频易错点

- 认为使用 protobuf 就等于 gRPC。
- 忽略 L7 LB/observability 对 HTTP/2 长连接的特殊处理。

动手验证 / 抓包实验

用 grpcurl 调一个本地服务，抓取 HTTP/2 stream，观察 headers、DATA 与 trailers。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

第八章 HTTPS、TLS 1.3 与 PKI

本章目标

掌握握手、证书、密钥协商与 0-RTT 风险。

建议：先能口述结论，再能画出关键时序/状态，最后通过命令或抓包验证。

74. HTTP 与 HTTPS 的本质区别是什么？TLS 提供了哪些安全属性？

难度：基础	频率：★★★★★	标签：HTTPS / TLS	定位：核心真题

30 秒标准回答

HTTPS 是 HTTP over TLS。TLS 主要提供机密性、完整性与对服务器身份的认证（也可做客户端证书认证）。它不能自动保证服务器业务本身可信，也不能阻止应用把敏感数据写入日志。

深度解析：Know-How + Know-Why

- HTTP 语义本身基本不因 TLS 改变。
- TLS 通过握手建立密钥，再用高效的对称 AEAD 保护应用数据。
- 证书认证的是域名/公钥绑定与信任链，不是“网站内容一定合法”。

面试官常见追问

- HTTPS 能隐藏域名吗？SNI/ECH 如何影响？
- TLS 能防 SQL 注入吗？
- 为什么 HTTPS 仍可能被企业代理解密？

高频易错点

- 说 HTTPS=HTTP+RSA。
- 认为上 HTTPS 就解决所有安全问题。

动手验证 / 抓包实验

用 `openssl s_client -connect host:443 -servername host` 查看证书链、TLS version、cipher 与 ALPN。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

75. TLS 1.3 Handshake 需要掌握到什么程度？

难度：进阶	频率：★★★★★	标签：TLS 1.3 / Handshake	定位：核心真题

30 秒标准回答

面试应能说明 ClientHello 携带版本、cipher suites、key share、SNI/ALPN 等；ServerHello 选择参数并给出 key share，双方基于 (EC)DHE 派生 handshake keys；随后服务器发送加密的证书/签名/Finished，客户端验证并返回 Finished，之后进入 application data。

核心时序 / 心智模型

```
ClientHello + key_share
          ↓
ServerHello + key_share
          ↓
Handshake keys derived
          ↓
Certificate / Verify / Finished
          ↓
Client Finished
          ↓
Encrypted Application Data
```

深度解析：Know-How + Know-Why

- TLS 1.3 大幅移除旧式静态 RSA key exchange 与许多不安全/复杂算法。
- ServerHello 后大部分握手内容已被加密。
- 证书签名用于身份认证，ECDHE 用于会话密钥协商，两者角色不同。

面试官常见追问

- TLS 1.2 与 1.3 往返和密码套件结构差异?
- CertificateVerify 在证明什么?
- Finished 如何证明握手未被篡改?

高频易错点

- 把服务器证书公钥直接当成最终 AES key。
- 把 cipher suite 中的签名算法理解成 TLS1.2 老模型。

动手验证 / 抓包实验

用 Wireshark 抓 TLS1.3 握手并标注 ClientHello、ServerHello、EncryptedExtensions、Certificate、Finished。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

76. 浏览器如何验证 HTTPS 证书？证书链、主机名、有效期各自是什么作用？

难度：进阶　**频率：**★★★★★　**标签：**PKI / Certificate Chain　**定位：**核心真题

30 秒标准回答

客户端从服务器证书向上构建到本地受信任根 CA 的链，验证每级签名、证书有效期、用途约束，并确认目标主机名与证书 SAN 匹配。浏览器信任的是本地 trust store 中的根，而不是服务器随便附带的“根证书”。

深度解析：Know-How + Know-Why

- 服务器通常发送 leaf + intermediate，根证书一般不需要发送。
- 现代主机名匹配以 Subject Alternative Name 为主。
- OCSP/CRL 等撤销机制补充“证书曾经合法但后来被撤销”的情况。

面试官常见追问

- 为什么中间证书配置错误会导致部分客户端失败?
- 自签名证书为什么会报错?
- 证书透明度 CT 解决什么?

高频易错点

- 认为只要证书没过期就可信。
- 把 Common Name 当成所有现代场景唯一主机名来源。

动手验证 / 抓包实验

用 `openssl s_client -showcerts` 导出链，逐级查看 Issuer/Subject/SAN/Validity，并用系统 CA 验证。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

77. TLS 为什么同时使用非对称密码学和对称加密？

难度：基础　**频率：**★★★★☆　**标签：**Key Exchange / AEAD　**定位：**核心真题

30 秒标准回答

非对称密码学适合身份认证、数字签名和建立共享密钥，但计算成本高；握手完成后双方派生对称 traffic keys，用 AES-GCM、ChaCha20-Poly1305 等 AEAD 高效加密大量应用数据。

深度解析：Know-How + Know-Why

- TLS 1.3 的 ECDHE 负责建立共享秘密，证书私钥通常用于签名认证，而不是“加密所有网页内容”。
- AEAD 同时提供保密与完整性，避免传统 MAC-then-encrypt 等复杂组合。
- 密钥派生会区分 client/server、handshake/application 等不同用途。

面试官常见追问

- RSA 签名与 RSA key exchange 有什么区别?
- 为什么不能直接把服务器公钥当 AES key?
- HKDF 在 TLS 1.3 里做什么?

高频易错点

- 说 HTTPS 用“RSA 加密 AES 密钥”作为 TLS1.3 标准流程。
- 把 hash、MAC、签名、加密混成一个概念。

动手验证 / 抓包实验

查看 TLS1.3 cipher suite 名称，解释为什么其中不再直接编码证书签名算法。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

78. 什么是 Forward Secrecy？为什么 ECDHE 很重要？

难度：进阶 **频率：**★★★★☆ **标签：**PFS / ECDHE **定位：**核心真题

30 秒标准回答

Forward Secrecy 指未来即使长期私钥泄露，也不应直接解密过去已完成会话。ECDHE 为每次会话生成临时 Diffie-Hellman 密钥材料，长期证书私钥只用于认证这些临时参数，不用于派生可长期复算的会话密钥。

深度解析：Know-How + Know-Why

- 前提是 ephemeral secret 被正确生成并销毁。
- 静态 RSA key exchange 一旦长期私钥泄露，历史录制流量可能被解密，因此 TLS1.3 已移除。
- 会话恢复/0-RTT 具有不同安全属性，需要单独分析。

面试官常见追问

- PFS 能否防当前实时 MITM?
- 如果客户端终端被攻陷，PFS 还能保护当前会话吗?
- ECDHE 中双方如何得到同一个共享秘密?

高频易错点

- 认为 PFS 等于“证书永不过期”。
- 认为只要算法名有 EC 就一定有 PFS。

动手验证 / 抓包实验

画出 ECDHE：双方各有 ephemeral private/public key，说明共享 secret 如何在不传私钥的情况下得到。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

79. SNI 与 ALPN 分别解决什么问题？

难度：进阶　**频率：**★★★★☆　**标签：**TLS Extension / Virtual Hosting　**定位：**核心真题

30 秒标准回答

SNI 在 TLS 握手早期告诉服务器客户端想访问哪个域名，使一个 IP 上可选择正确证书/虚拟主机；ALPN 让双方协商 TLS 上层应用协议，例如 `h2` 或 `http/1.1`。一个解决“哪个主机”，一个解决“说哪种协议”。

深度解析：Know-How + Know-Why

- 传统 SNI 通常可被路径观察者看到，ECH 旨在加密 ClientHello 中的敏感部分。
- ALPN 是 HTTP/2 over TLS 部署的关键协商机制。
- 同一证书可覆盖多个 SAN，所以 SNI 与证书并非一对一。

面试官常见追问

- 没有 SNI 时共享 IP HTTPS 如何工作?
- ALPN 协商失败会怎样?
- ECH 是否隐藏目标 IP?

高频易错点

- 把 SNI 当作 DNS。
- 把 ALPN 说成 TLS cipher negotiation。

动手验证 / 抓包实验

用 `openssl s_client -servername` 对同一 CDN IP 测不同域名；查看 ALPN negotiated protocol。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

80. TLS 1.3 0-RTT 为什么有 Replay Risk？哪些业务不适合？

难度：进阶	频率：★★★★★	标签：0-RTT / Replay / Idempotency	定位：核心真题

30 秒标准回答

0-RTT early data 在完整新握手确认前即可发送，攻击者可能复制并重放这段合法密文到服务器。即使攻击者不知道明文，也可能让非幂等操作被执行多次，因此支付、下单、发券、状态变更等操作通常不应无保护地接受 0-RTT。

深度解析：Know-How + Know-Why

- 服务端 anti-replay 机制在多机/多区域下实现并不简单。
- 0-RTT 的目标是省一个 RTT，而不是提高带宽。
- 应用可只允许安全/幂等请求进入 early data，或要求幂等键。

面试官常见追问

- 为什么 GET 也不一定绝对 safe?
- 多数据中心如何共享 anti-replay 状态?
- QUIC 0-RTT 与 TLS 0-RTT 是什么关系?

高频易错点

- 说 0-RTT “没有加密”。
- 只依赖 HTTP method 名称判断是否绝对安全。

动手验证 / 抓包实验

列出你系统中的 API，分类哪些允许 early data，哪些必须等待 1-RTT，并说明幂等设计。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

第九章 Socket、epoll 与 Linux 网络栈

本章目标

把协议知识落到系统调用、事件循环与内核收包路径。

建议：先能口述结论，再能画出关键时序/状态，最后通过命令或抓包验证。

81. 一个 TCP Server 从 socket() 到处理请求的系统调用流程是什么？

难度：基础　频率：★★★★★　标签：Socket API / Server　定位：核心真题

30 秒标准回答

服务端典型流程是 socket→setsockopt→bind→listen→accept→recv/read→send/write→close；监听 socket 只负责接收新连接，accept 返回新的 connected socket，每个连接拥有独立状态。高并发服务通常再配合非阻塞 IO 与事件循环。

深度解析：Know-How + Know-Why

- bind 把本地地址/端口绑定到 socket。
- listen 把 socket 变成被动监听状态并建立连接队列。
- accept 取出已完成握手的连接，不负责执行 TCP 三次握手本身。

面试官常见追问

- accept4 有什么用?
- SO_REUSEADDR 与 SO_REUSEPORT 区别?
- 一个 listen fd 能被多个进程共享吗?

高频易错点

- 认为 accept 返回的还是监听 fd。
- 把 listen(backlog) 当成业务最大并发数。

动手验证 / 抓包实验

写最小 C/Python TCP server，用 `strace` 观察 socket/bind/listen/accept/read/write 调用。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

82. Blocking Socket 与 Non-blocking Socket 的核心区别是什么？

难度：基础 **频率：**★★★★★ **标签：**Blocking / EAGAIN **定位：**核心真题

30 秒标准回答

阻塞 socket 在当前操作无法继续时会让调用线程睡眠等待；非阻塞 socket 会立即返回 `EAGAIN/EWOULDBLOCK`（connect 可能 EINPROGRESS），由应用稍后根据 readiness 再尝试。非阻塞本身不会自动提高性能，通常要与事件驱动架构结合。

深度解析：Know-How + Know-Why

- readiness 表示“现在尝试大概率不会阻塞”，不是数据永远够用。
- 非阻塞 send 可能只写入部分数据，必须维护剩余 buffer。
- 连接建立、读、写、accept 都有各自的非阻塞状态处理。

面试官常见追问

- nonblocking connect 如何判断成功?
- 为什么 write-ready 事件通常不应一直订阅?
- EAGAIN 与 EOF 如何区分?

高频易错点

- 把 nonblocking 理解成“异步 IO”。
- 忽略 partial write。

动手验证 / 抓包实验

将 socket 设置 O_NONBLOCK，故意填满 send buffer，观察 send 返回部分写/EAGAIN。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

83. 经典五种 I/O Model 是什么？readiness 与 completion 的区别是什么？

难度：进阶 **频率：**★★★★☆ **标签：**IO Model / AIO **定位：**核心真题

30 秒标准回答

经典教材常分为阻塞 IO、非阻塞 IO、IO 多路复用、信号驱动 IO、异步 IO。关键是区分“等待数据就绪”和“把数据拷贝到用户缓冲”两个阶段；epoll 主要通知 readiness，而真正 completion-based 异步 IO 会在操作完成后通知。

深度解析：Know-How + Know-Why

- select/poll/epoll 都属于 readiness-based multiplexing。
- Linux io_uring 可覆盖更多 completion 风格操作，但具体语义取决于操作与模式。
- 事件循环中仍需执行 recv/send，因此“epoll 异步读取了数据”是不准确表达。

面试官常见追问

- Reactor 与 Proactor 有什么区别？
- io_uring 为什么对文件 IO 更有意义？
- 为什么信号驱动 IO 实际网络服务中不常见？

高频易错点

- 把 epoll 直接称为 AIO。
- 只背五个名字，不讲两个阶段。

动手验证 / 抓包实验

画出 blocking、epoll、io_uring 三种路径，标明谁等待、谁执行 copy、谁收到通知。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

84. select、poll、epoll 的核心差异是什么？

难度：进阶 **频率：**★★★★★ **标签：**IO Multiplexing **定位：**核心真题

30 秒标准回答

select 通过位图 fd_set 传递关注集合并有接口级 fd 数限制；poll 用 pollfd 数组改善接口但每轮仍需处理整个集合；epoll 把 interest set 维护在内核，`epoll_wait` 返回就绪事件，更适合“连接很多、活跃较少”的长期事件驱动场景。

深度解析：Know-How + Know-Why

- epoll 的优势不应粗暴概括成“时间复杂度永远 O(1)”。
- 连接数很少时三者差距可能不重要，架构复杂度反而更重要。
- epoll 注册、事件分发、用户处理和 cache locality 都有实际成本。

面试官常见追问

- epoll_ctl 为什么不是每轮都调用？
- poll 在何种场景足够好？
- kqueue 与 epoll 思路有何相似？

高频易错点

- 背“select 1024、poll O(n)、epoll O(1)”就结束。
- 认为 epoll 自动创建线程。

动手验证 / 抓包实验

写同一个 echo server 的 poll/epoll 版本，用 1k/10k idle connections 比较系统调用和 CPU。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

85. epoll LT 与 ET 有什么区别？ET 为什么要求“读到 EAGAIN”？

难度：进阶 **频率：**★★★★★ **标签：**epoll / LT / ET **定位：**核心真题

30 秒标准回答

LT 只要 fd 仍处于可读/可写状态就会持续报告；ET 更强调状态从未就绪到就绪的边沿变化，因此处理一次 EPOLLIN 时通常要循环 read，直到返回 EAGAIN，确保把当前可读数据 drain 完，否则剩余数据可能长时间没有新的边沿提醒。

深度解析：Know-How + Know-Why

- ET 可减少重复事件但增加状态机复杂度。
- 对写事件同理：应尽量 flush 到 EAGAIN，再根据剩余 buffer 决定是否继续订阅 EPOLLOUT。
- 无论 LT/ET 都必须正确处理 EOF、ERR、HUP。

面试官常见追问

- ET 下只读一次会发生什么?
- EPOLLONESHOT 解决什么?
- 为什么 ET 不必然更快?

高频易错点

- 把 ET 解释成“只通知一次，永远不再通知”。
- 没有处理 partial read/write。

动手验证 / 抓包实验

构造一次写入 1MB 的客户端，ET server 每次只读 1KB，观察未 drain 时事件行为，再修复为循环到 EAGAIN。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

86. 为什么 epoll ET 几乎总与 nonblocking socket 搭配？

难度：进阶 **频率：**★★★★★ **标签：**ET / Nonblocking **定位：**核心真题

30 秒标准回答

ET handler 通常要循环操作直到当前缓冲被读空/写满；若 fd 是阻塞模式，最后一次 read 在已经没有数据时可能把整个 event-loop 卡住。非阻塞使循环能以 EAGAIN 作为“当前已 drain 完”的可靠退出条件。

深度解析：Know-How + Know-Why

- 这是 correctness 要求，不只是性能优化。
- 多线程 event loop 也可能因一个阻塞 fd 占住 worker 导致大面积 tail latency。
- accept 也常循环 accept4 到 EAGAIN，以处理一批新连接。

面试官常见追问

- LT 是否也推荐 nonblocking?
- 为什么不能用 ioctl(FIONREAD) 代替循环 read?
- ET 下如何处理 SSL_read?

高频易错点

- 认为 ET+blocking 只是“慢一点”。
- 忘记 accept fd 自身也应设 nonblocking。

动手验证 / 抓包实验

故意用 ET+blocking server，在 handler 循环 read，复现卡死；切到 O_NONBLOCK 后观察 EAGAIN。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

87. 为什么 epoll 对普通磁盘文件意义有限？

难度：进阶 **频率：**★★★☆☆ **标签：**epoll / File IO **定位：**核心真题

30 秒标准回答

普通文件通常被视为“总是可读/可写”的对象，epoll 的 readiness 不能代表底层磁盘数据已经无需等待；真正读文件仍可能触发 page fault、I/O 等待。高并发文件 IO 更适合线程池、AIO 或 io_uring 等机制。

深度解析：Know-How + Know-Why

- 网络 socket 的 readiness 来自异步到达数据，普通文件语义不同。
- page cache 命中时 read 很快，不命中时可能等待存储。
- sendfile 能优化文件→socket 数据路径，但也不等于磁盘永不阻塞。

面试官常见追问

- io_uring 与 epoll 的抽象差异?
- 为什么 regular file 不能简单 add 到 epoll?
- Direct I/O 会怎样改变文件读取路径?

高频易错点

- 把 epoll 当成万能“异步所有 fd”。
- 认为 page cache 命中/未命中对延迟无影响。

动手验证 / 抓包实验

对一个大文件 cold-cache 读取用 `perf`/`iostat` 观察等待；与网络 epoll readiness 对比。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

88. send() 返回成功，是否意味着对端已经收到或处理？

难度：工程	频率：★★★★★	标签：send / Kernel Buffer / App ACK	定位：核心真题

30 秒标准回答

不意味着。send 成功通常表示内核接受了这部分用户数据进入本地 socket 发送路径；之后可能还要排队、分段、发包、丢失、重传。即便收到 TCP ACK，也只证明对端 TCP stack 已确认字节，不证明对端应用已 read，更不证明业务事务已提交。

深度解析：Know-How + Know-Why

- nonblocking send 还可能只接受一部分数据。
- 连接随后 reset 时，应用甚至可能无法准确知道最后哪些业务请求已执行。
- Exactly-once 业务语义通常靠 request ID、幂等键、事务与应用 ACK 设计。

面试官常见追问

- write() 返回后进程立刻 crash 会怎样?
- TCP ACK 与数据库 commit 的时间关系?
- 如何设计支付请求重试?

高频易错点

- 把 `send()==len` 当成“对端收到”。
- 用 TCP 可靠性替代业务幂等。

动手验证 / 抓包实验

设计客户端 send 后立刻断网的实验，让服务端记录是否实际接收；讨论客户端如何判断执行结果。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

89. Zero Copy 的目标是什么？sendfile、splice、mmap 各自大致减少了什么？

难度：进阶 **频率：**★★★★☆ **标签：**Zero Copy / sendfile **定位：**核心真题

30 秒标准回答

Zero Copy 的核心是减少不必要的用户态/内核态数据复制、上下文切换与 CPU 搬运。`sendfile` 可让文件数据在内核中直接进入 socket 发送路径；`splice` 在支持的 fd 间通过 pipe buffer 转移引用；mmap 让用户映射 page cache，但仍需理解 page fault 与写 socket 的路径。

深度解析：Know-How + Know-Why

- “Zero”不是物理意义完全 0 次移动，NIC/DMA/内存总线仍会搬数据。
- sendfile 非常适合静态文件服务，但业务需要修改/加密用户数据时路径会更复杂。
- TLS zero-copy 需要额外内核/硬件支持，不能默认与纯 sendfile 等价。

面试官常见追问

- DMA 在 zero-copy 中起什么作用?
- 为什么 HTTPS 静态文件比 HTTP sendfile 更复杂?
- copy_file_range 与 sendfile 有何不同定位?

高频易错点

- 把 mmap 直接称为真正 0 copy。
- 认为 zero-copy 唯一收益是带宽。

动手验证 / 抓包实验

用 `strace`/`perf stat` 对比 read+write 与 sendfile 发送大文件的 syscalls、CPU cycles。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

90. 一个 Packet 从 NIC 到应用 recv() 大致经过哪些 Linux 网络栈阶段？

难度：高级　**频率：**★★★★☆　**标签：**NIC / NAPI / Softirq / Socket Queue　**定位：**核心真题

30 秒标准回答

典型接收路径：NIC 收包写入 RX ring/DMA→中断或 NAPI poll→driver 构造 skb→二层/IP→Netfilter/路由→TCP/UDP→socket receive queue→唤醒/事件通知→应用 recv。多核场景还涉及 RSS、RPS、RFS、IRQ affinity 和 NUMA。

核心时序 / 心智模型

```
NIC RX Ring (DMA)
      ↓
IRQ / NAPI poll
      ↓
Driver + skb
      ↓
IP / TCP / Netfilter
      ↓
Socket Receive Queue
      ↓
epoll/readiness
      ↓
Application recv()
```

深度解析：Know-How + Know-Why

- NAPI 用中断触发+轮询缓解高包率下 interrupt storm。
- RSS 由 NIC 按 hash 把 flow 分散到硬件 RX queues；RPS/RFS 是软件侧进一步分发。
- 高 PPS 场景 CPU 可能主要耗在 softirq，而业务线程看起来并不忙。

面试官常见追问

- RSS 与 RPS 有什么区别?
- 为什么单队列 NIC 会卡单核?

- GRO/LRO 如何减少 per-packet 开销?

高频易错点

- 只说“网卡→内核→应用”，没有队列与软中断视角。
- 认为包一定在接收中断上下文完成全部 TCP 处理。

动手验证 / 抓包实验

观察 `/proc/interrupts`、`/proc/softirqs`、`ethtool -l/-S`，结合压测定位 RX queue 与 CPU 分布。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

第十章 综合故障定位与系统设计

本章目标

用分层方法处理真实线上问题与高并发架构题。

建议：先能口述结论，再能画出关键时序/状态，最后通过命令或抓包验证。

91. 浏览器输入 https://www.example.com 到页面显示，完整发生了什么？

难度：综合 **频率：**★★★★★ **标签：**URL 全流程 / 综合 **定位：**核心真题

30 秒标准回答

优秀答案应按层展开：URL 解析→DNS→路由/ARP(NDP)→本地网关/NAT→互联网/CDN/LB→TCP 或 QUIC 建连→TLS/证书/ALPN→HTTP 请求→反向代理/应用/数据库→响应→缓存/解压→浏览器 HTML/CSS/JS 解析与渲染。面试官在哪层追问，就向该层深入。

核心时序 / 心智模型

```
URL
 ↓ DNS
IP + Route + ARP/NDP
 ↓
TCP/QUIC
 ↓
TLS + ALPN
 ↓
HTTP
 ↓
CDN / LB / App
 ↓
Response
 ↓
Browser Parse & Render
```

深度解析：Know-How + Know-Why

- 不要把流程背成固定 20 步，要能依据 HTTP/1.1、HTTP/2、HTTP/3 分支。
- 如果命中 CDN edge，可能根本不到源站；如果缓存命中浏览器，也可能不发网络请求。
- 现代浏览器可能并行 DNS、预连接、0-RTT、资源预加载等。

面试官常见追问

- 如果 DNS 已缓存，哪些步骤消失?
- HTTP/3 与 HTTP/2 的建连差异?
- 页面白屏但 network 200，问题可能在哪?

高频易错点

- 漏掉 DNS、TLS 或缓存任何一大块。
- 说“TCP 三次握手后直接渲染”，忽略 HTTP 与浏览器处理。

动手验证 / 抓包实验

选一个网页做完整 waterfall：DevTools + tcpdump + dig + curl timing，给每阶段标注时间。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

92. 用户说“网站突然很慢”，你如何系统化拆解而不是猜？

难度：工程 **频率：**★★★★★ **标签：**Latency / Troubleshooting **定位：**核心真题

30 秒标准回答

先把总延迟拆成 DNS、connect、TLS、TTFB、download、client rendering，再定位网络 RTT/loss、CDN hit、LB、应用、数据库和前端。建立分层时间线比“先重启服务器”更可靠。

深度解析：Know-How + Know-Why

- `curl -w` 可快速分解 name lookup/connect/appconnect/starttransfer/total。
- TTFB 高偏向服务器/上游处理或网络往返；download 高偏向响应体、带宽、丢包。
- 浏览器慢但 curl 快，可能是 JS、主线程、资源依赖和渲染。

面试官常见追问

- 怎么判断 DNS 慢还是 connect 慢?
- p99 上升但 p50 正常通常看什么?
- CDN miss storm 会怎样体现?

高频易错点

- 只看 CPU/QPS。
- 只用 ping 判断整个网站性能。

动手验证 / 抓包实验

为一个请求同时收集 curl timing、traceroute/mtr、应用 tracing、DB slow query，构建 waterfall。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

93. 服务延迟升高但 CPU、QPS 都没变化，你优先怀疑什么？

难度：工程　**频率：**★★★★★　**标签：**Tail Latency / Queueing　**定位：**核心真题

30 秒标准回答

应考虑 CPU 之外的等待：网络 RTT/packet loss/retransmission、DNS、连接池耗尽、锁/线程池、磁盘/DB、GC、跨 AZ 路由、NIC/softirq、下游 timeout、队列排队等。系统性能问题通常是“等待在哪里”，而非只看计算量。

深度解析：Know-How + Know-Why

- 丢包很少也可能显著放大 p99，因为重传至少增加一个 RTT 或 RTO。
- 连接池耗尽会让请求在应用入口排队，CPU 反而可能下降。
- 路由变化可能让 RTT 突然增加但应用指标看起来正常。

面试官常见追问

- 如何用 Little’s Law 理解队列堆积?
- 怎样从 retrans counter 判断网络丢包?
- 跨 AZ 流量为什么会引入 tail latency?

高频易错点

- CPU 不高就排除服务器问题。
- 看到 RTT 高就直接归因公网。

动手验证 / 抓包实验

对比事故前后 `ss -s` retrans、TCP RTT、连接池等待、线程池 queue、DB latency 与路由路径。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

94. 大量 TIME_WAIT 的完整排障流程是什么？

难度：工程　**频率：**★★★★★　**标签：**TIME_WAIT / Connection Churn　**定位：**核心真题

30 秒标准回答

从“谁主动关闭”出发，统计连接创建/关闭速率、目的端点、连接池命中、HTTP keepalive、部署/健康检查、ephemeral ports、NAT/LB 表项，再决定是修复连接复用还是调整系统边界。数量本身不是唯一问题，速率与资源上限更关键。

深度解析：Know-How + Know-Why

- 同样 100k TIME_WAIT，在百万连接/秒系统和千 QPS 系统含义完全不同。
- 客户端端口耗尽会表现为 connect 错误，而不是简单 CPU 高。
- 代理 upstream keepalive 未配置是非常常见根因。

面试官常见追问

- 如何计算理论 ephemeral port 周转压力?
- 为什么多目标 IP 能缓解单四元组端口冲突?
- HTTP/2 复用对短连接 churn 有何帮助?

高频易错点

- 事故时立即开启危险的 TIME_WAIT recycle 类参数。
- 只看总数，不看创建速率。

动手验证 / 抓包实验

输出 `ss` 统计、连接创建 rate、top remote endpoints、端口范围、LB/NAT 指标，形成一页诊断表。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

95. 大量 CLOSE_WAIT 的完整排障流程是什么？

难度：工程　**频率：**★★★★★　**标签：**CLOSE_WAIT / Application Leak　**定位：**核心真题

30 秒标准回答

先用 `ss -tanp state close-wait` 找 PID/远端与数量，确认对端已正常 FIN；再检查应用未 close 的代码路径、异常处理、HTTP body 未关闭、连接池、线程阻塞和 fd 泄漏。长期 CLOSE_WAIT 几乎总需要修业务资源生命周期。

深度解析：Know-How + Know-Why

- 监控 FD 数与 CLOSE_WAIT 同步增长可强化泄漏判断。
- 重启只能释放资源，不会修复根因。

- Go 的 http.Response.Body、Java stream/connection、C++ RAII 都有典型坑。

面试官常见追问

- 如何用 eBPF 跟踪 socket close 生命周期?
- 连接池中“归还”与真正 close 的关系?
- 对端 RST 时还会进入 CLOSE_WAIT 吗?

高频易错点

- 调 `tcp_fin_timeout` 试图解决 CLOSE_WAIT。
- 只找网络团队，不检查应用。

动手验证 / 抓包实验

结合 lsof、线程 dump、heap/resource trace 找到最早创建但未关闭的 socket 调用路径。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

96. 大量 SYN_RECV 且新连接失败，应如何判断是攻击、丢包还是容量问题？

难度：工程 **频率：**★★★★★ **标签：**SYN_RECV / Backlog **定位：**核心真题

30 秒标准回答

同时看 SYN arrival rate、SYN+ACK retrans、半连接队列、syncookie counters、NIC/drop、LB、防火墙、CPU softirq 与正常流量基线。如果握手完成率下降且源地址异常分散，可能是攻击；若链路丢包高则可能正常客户端 ACK 回不来；若 accept queue 满则是另一个应用问题。

深度解析：Know-How + Know-Why

- 半连接与全连接队列要分别判断。
- SYN flood 可用伪造源地址，而真实流量高峰通常能看到完整握手。
- 边缘 LB/防火墙也可能先成为瓶颈，源主机指标看不到全部 SYN。

面试官常见追问

- 如何从抓包判断 SYN+ACK 是否离开服务器?
- syncookies counter 说明什么?
- 为什么 SYN backlog 调大也可能治标不治本?

高频易错点

- 看到 SYN_RECV 就只会说 DDoS。
- 把 accept 慢与握手失败混为一谈。

动手验证 / 抓包实验

构造诊断矩阵：SYN rate、SYN-ACK retrans、ACK rate、listen queue、syncookie、drop counters，对每种组合给出结论。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

97. 网络服务 CPU 很高但业务线程不忙，可能是什么？

难度：高级 **频率：**★★★★☆ **标签：**Softirq / PPS / NIC **定位：**核心真题

30 秒标准回答

优先看软中断、每秒包数、NIC RX/TX queues、iptables/nft/conntrack、GRO、重传和 IRQ affinity。大量小包会使 CPU 花在 per-packet 协议处理上，即使业务 QPS 不高；单 RX queue 还可能形成单核热点。

深度解析：Know-How + Know-Why

- Mbps 低不代表 PPS 低，64B tiny packet 可以打满 CPU。
- `ksoftirqd` 高、NET_RX softirq 高是网络栈热点线索。
- RSS/RPS/XPS、GRO/GSO、irqbalance 都会影响多核扩展。

面试官常见追问

- 如何区分包率问题与 TLS 加密问题?
- conntrack 表满/锁竞争会怎样?
- NUMA 跨节点内存访问为何影响 NIC 性能?

高频易错点

- 只看 top 中业务进程 CPU。
- 用带宽 Mbps 代替 PPS 判断网络负载。

动手验证 / 抓包实验

收集 `mpstat -P ALL`、`/proc/softirqs`、`ethtool -S`、pps、perf top，定位 CPU 花在哪里。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

98. 四层负载均衡与七层负载均衡怎么选？

难度：系统设计　**频率：**★★★★★　**标签：**L4 / L7 / Load Balancing　**定位：**核心真题

30 秒标准回答

L4 基于 IP/Port/协议转发，通用、开销低、可承载任意 TCP/UDP；L7 理解 HTTP/gRPC 等应用协议，可按 Host/Path/Header/Cookie 路由、做认证、重试、限流和可观测，但需要解析/终止协议并承担更多 CPU、内存与语义复杂度。

深度解析：Know-How + Know-Why

- TLS passthrough 更像 L4；TLS termination 后才能做多数 HTTP L7 路由。
- L7 retry 必须理解请求幂等性，否则可能重复副作用。
- 长连接和 HTTP/2 connection pooling 会让“连接均衡”与“请求均衡”出现差异。

面试官常见追问

- NAT mode、DSR、proxy mode 有何区别?
- L7 LB 为什么可能产生 502/504?
- 一致性哈希适合什么场景?

高频易错点

- 说 L7 一定比 L4 更先进。
- 忽略 TLS termination 对证书与安全边界的影响。

动手验证 / 抓包实验

画出 Client→L4 LB→L7 Proxy→Service 的链路，对每层列出可见字段、状态、故障码和监控。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

99. 设计支持 100 万 TCP 长连接的服务器，第一性约束有哪些？

难度：系统设计　**频率：**★★★★★　**标签：**Million Connections / Capacity Planning　**定位：**核心真题

30 秒标准回答

先做资源模型而不是先说 epoll：FD 上限、每连接内核/用户内存、socket buffers、心跳包率、TLS 状态、event-loop 分片、accept/worker 架构、RSS/多核、NAT/LB、连接迁移与滚动发布。百万连接的关键常是内存与 PPS，而非纯连

接计数。

深度解析：Know-How + Know-Why

- 若每连接总状态 20KB，100 万连接就是约 20GB，还没算 page cache、应用和内核额外开销。
- 30 秒一次心跳意味着约 33k heartbeat/s；若双向、多包，PPS 更高。
- 滚动发布要 connection draining，否则一次重启会制造重连风暴。

面试官常见追问

- SO_REUSEPORT 如何分摊监听？
- 如何做长连接负载均衡和会话迁移？
- 为什么连接数大但活跃率低仍可能内存很高？

高频易错点

- 只回答“epoll 能支持百万连接”。
- 忽略客户端/网关端口和 NAT 表限制。

动手验证 / 抓包实验

建立容量表：FD、bytes/conn、heartbeat PPS、TLS memory、CPU/packet、accept rate、reconnect storm，并做 2x safety margin。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

---

100. 给你一份 tcpdump/Wireshark 抓包，如何系统判断 TCP 问题？

难度：综合	频率：★★★★★	标签：Packet Analysis / Troubleshooting	定位：核心真题

30 秒标准回答

先按连接生命周期看握手、数据、关闭，再观察 retransmission、duplicate ACK、SACK、out-of-order、zero window、RST、RTT、MSS、window scaling、ECN。不要依赖 Wireshark 的单一“Expert Info”标签，要结合序号、时间线和两端抓包判断丢包发生在哪。

深度解析：Know-How + Know-Why

- 只有客户端抓包看到 Retransmission，不代表一定在客户端到服务器方向真实丢失，也可能抓包点丢包。
- SYN 重传而无 SYN+ACK：路由、防火墙、服务未监听、丢包都可能。
- 握手正常但长时间无 application response，应转向 HTTP/upstream/应用处理。

面试官常见追问

- 如何区分真正 packet loss 与 packet capture drop?
- RST 是谁发的？如何从 TTL/IP/seq 推断?
- Zero Window 与 Window Full 分别是什么?

高频易错点

- 看到“TCP Retransmission”就直接认定网络运营商丢包。
- 只看单包，不画时间序列。

动手验证 / 抓包实验

选取一个故障 pcap，制作“时间戳-方向-Seq/Ack-Flags-Len-解释”表，最终给出证据链和不确定项。

评分标准

及格：能给出核心定义与基本流程。

良好：能解释关键状态/字段，并回答至少 2 个追问。

优秀：能说明设计取舍、异常路径，并给出 Linux/抓包证据。

附录 A：Top 20 必会题与 7 天冲刺顺序

Day 1

- 22. TCP 三次握手完整过程是什么？每一步同步了什么状态？
- 23. 为什么 TCP 需要三次握手，不能两次？
- 25. TCP 为什么通常需要四次挥手？FIN 与 ACK 的语义是什么？

Day 2

- 26. TIME_WAIT 为什么存在？为什么经常讨论 2MSL？
- 28. CLOSE_WAIT 大量堆积通常说明什么？
- 32. TCP 可靠性由哪些机制共同保证？

Day 3

- 34. TCP 滑动窗口解决什么问题？发送窗口为什么不是固定大小？
- 36. Flow Control 与 Congestion Control 的区别是什么？
- 39. rwnd、cwnd、ssthresh 三者是什么关系？

Day 4

- 44. BDP 是什么？为什么高带宽长 RTT 链路需要大窗口？
- 53. 输入域名后，DNS 完整解析链路是什么？
- 61. HTTP/1.1 Request/Response 报文结构是什么？

Day 5

- 67. Cache-Control 中 max-age、no-cache、no-store、private、must-revalidate 怎么理解？
- 69. HTTP/1.1 与 HTTP/2 的核心差异是什么？
- 70. HTTP/2 为什么仍存在 Head-of-Line Blocking？

Day 6

- 74. HTTP 与 HTTPS 的本质区别是什么？TLS 提供了哪些安全属性？
- 75. TLS 1.3 Handshake 需要掌握到什么程度？
- 84. select、poll、epoll 的核心差异是什么？

Day 7

- 85. epoll LT 与 ET 有什么区别？ET 为什么要求“读到 EAGAIN”？
- 91. 浏览器输入 https://www.example.com 到页面显示，完整发生了什么？

冲刺原则

Day 1-3：TCP 状态机与可靠性。

Day 4：DNS/URL 全流程。

Day 5：HTTP/2/3 + TLS。

Day 6：epoll/Linux 网络栈。

Day 7：只做抓包与故障题，不再背新知识。

附录 B：Linux 网络排障命令速查

目的	命令	主要看什么
连接/状态	`ss -s; ss -tanp; ss -lnt; ss -ti`	连接数、TCP 状态、监听队列、RTT/cwnd
路由	`ip route; ip route get <dst>`	默认路由、最长前缀匹配结果
邻居	`ip neigh`	ARP/NDP 邻居缓存
DNS	`dig; dig +trace; resolvectl`	递归结果、权威链路、Resolver 状态
抓包	`tcpdump -i any -nn -s0 -w out.pcap`	生成可供 Wireshark 分析的证据
链路	`ping; traceroute; mtr`	RTT、路径与丢包线索
NIC	`ethtool -S eth0; ethtool -l eth0`	drop/error、队列能力
CPU 网络	`cat /proc/softirqs; cat /proc/interrupts`	NET_RX/IRQ/队列热点
连接跟踪	`conntrack -S / conntrack -L`	NAT/conntrack 容量与状态
HTTP	`curl -w ... -v`	DNS/connect/TLS/TTFB/total 分解

附录 C：权威协议与实现参考

- TCP：RFC 9293 - Transmission Control Protocol
- TCP Congestion Control：RFC 5681 - TCP Congestion Control
- CUBIC：RFC 9438 - CUBIC for Fast Long-Distance Networks
- IPv6：RFC 8200 - Internet Protocol, Version 6
- Neighbor Discovery：RFC 4861 - Neighbor Discovery for IPv6
- DNS Concepts：RFC 1034 / RFC 1035
- Negative DNS Cache：RFC 2308
- HTTP Semantics：RFC 9110
- HTTP Caching：RFC 9111
- HTTP/2：RFC 9113
- QUIC：RFC 9000
- HTTP/3：RFC 9114
- TLS 1.3：RFC 8446
- BGP-4：RFC 4271
- Linux 网络实现：Linux Kernel Documentation: networking/、ip-sysctl、scaling

阅读顺序

不要从 RFC 第一页顺读。先带着具体问题去查：

“TIME_WAIT 的状态语义是什么？” → TCP RFC。

“HTTP/2 为什么仍受 TCP HOL？” → HTTP/2 + TCP。

“为什么 ET 要 read 到 EAGAIN？” → epoll man page + 最小实验。

协议文档用来校正边界条件，实验用来建立直觉。

附录 D：面试自评表

等级	判定标准
1 分	只会名词，不能画时序/状态。
2 分	能背主流程，但异常/Why 追问会断。
3 分	能解释字段、状态、常见异常。
4 分	能结合 Linux、抓包、性能取舍。
5 分	能从业务现象反推协议/内核证据，并主动指出边界与不确定性。

End of Book · 100 Questions Completed